A Day at DIU: A Six-Scene Real-Time 2D/3D OpenGL Animation of a Student's Campus Journey

Submitted By

Student Name	Student ID
Tanvir Hossain Jisan	[232-15-870]
A.Q.Mohiuddin Mahim	[232-15-869]
Afrose Jahan Urmi	[232-15-784]
Modabberul Islam	[232-15-911]
Shakibul Islam Naeem	[232-15-202]

LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course

CSE422: Computer Graphics Lab in the Department of Computer Science and Engineering (CSE)



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

[11/08/2026]

Declaration

We hereby declare that this lab project has been done by us under the supervision of **Md Shah Jalal**, **Senior Lecturer**, Department of Computer Science and Engineering (CSE), Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as a lab project or coursework.

Submitted To:

_____

[MD.SHAH JALAL]
[Senior Lecturer]
Department of Computer Science and Engineering (CSE)
Daffodil International University

Submitted by

_____ [Tanvir Hossain Jisan] Student ID: [232-15-870] Dept. of CSE, DIU	_____ [A.Q.Mohiuddin Mahim] Student ID: [232-15-869] Dept. of CSE, DIU
_____ [Afrose Jahan Urmi] Student ID: [232-15-784] Dept. of CSE, DIU	_____ [Modabberul Islam] Student ID: [232-15-911] Dept. of CSE, DIU
_____ [Shakibul Islam Naeem] Student ID: [232-15-202] Dept. of CSE, DIU	

Course & Program Outcome

The course associated with this project has the following course outcomes:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Understand computer graphics systems and implement graphics primitives (lines, circles, polygons) for constructing a real-time animated scene.
CO2	Apply appropriate OpenGL/GLUT programming techniques, resources, and modern engineering tools to solve graphics-programming problems involving multiple shapes, 2D and 3D transformations.
CO3	Perform effectively as an individual or as a member of a team, through proper documentation, task division, and initialization of project work.
CO4	Create a multi-scene project by explaining complex computer engineering activities to the computer engineering community through effective reports, design documentation, and clear technical presentation.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

Mapping Course Outcomes (COs) with the Teaching-Learning and Assessment Strategy:

CO's	Corresponding PO number	Domain Level/ Learning Taxonomy	Level of Knowledge Profile (K)	Complex Engineering Problem (EP)	Complex Engineering Activities (EA)
CO1	PO1	C1, C2	K1-K4	EP1	-
CO2	PO5	C3, C4	K1-K4, K6	EP1, EP3	-
CO3	PO9	C4, A2	K6	EP1, EP3	EA1
CO4	PO10	C6, P3, A2	K4	EP3, EP5	EA2, EA3

The mapping justification of this table is provided in sections 4.2.1, 4.2.2, and 4.2.3.

Table of Contents

Submitted By 1
Declaration 2
Course & Program Outcome 3
Table of Contents 4
Introduction 6
1.1 Introduction 6
1.2 Motivation 6
1.3 Objectives 7
1.4 Feasibility Study 7
1.5 Gap Analysis 7
1.6 Project Outcome 8
System Architecture / Scene Description & Architecture 10
2.1 Requirement Analysis & Design Specification 10
2.1.1 Scene Overview 10
2.1.2 Object Components 14
2.2 Use of Modern Tools 14
2.3 Algorithms Used 15
2.4 2D Transformations Applied 16
2.5 Animation Logic 17
Implementation and Results 18
3.1 Implementation 18
Scene Construction & Layering 18
Implementation Details of the Core Rasterization Principles 18
Object Design and Modular Functions 19
3.2 Output & Screenshots 19
Scene 1: Morning Alarm in a Celebrity's Room 20
Scene 2: Student Goes to the Bus Stop 21
Scene 3: DIU Student Bus Journey 22
Scene 4: DIU Campus Arrival 24
Scene 5: Elevator Ride to the Classroom Floor 26
Scene 6: 3D DIU Classroom 27
3.3 Discussion 29
Strengths 29

Challenges Faced 29
Solutions 29
Engineering Standards and Mapping 31
4.1 Impact on Society, Environment and Sustainability 31
4.1.1 Impact on Life 31
4.1.2 Impact on Society & Environment 31
4.1.3 Ethical Aspects 31
4.1.4 Sustainability Plan 32
4.2 Complex Engineering Problem 32
4.2.1 Mapping of Program Outcome 32
4.2.2 Complex Problem Solving 32
4.2.3 Complex Engineering Activities 33
Conclusion 35
5.1 Summary 35
5.2 Limitations 35
5.3 Future Work 36
Hand Drawing Graph 38

Chapter 1

Introduction

1.1 Introduction

Computer Graphics is a branch of computer science concerned with the creation, manipulation, and representation of visual images using computers. It plays a vital role in modern applications such as gaming, simulation, virtual reality, education, and animation.

OpenGL (Open Graphics Library) is a widely used, cross-platform graphics application programming interface (API) that allows developers to render both 2D and 3D graphics efficiently. Together with the GLUT (OpenGL Utility Toolkit) windowing layer, it provides a set of low-level functions to draw primitives such as points, lines, polygons, and solids -- the fundamental building blocks of every graphical scene -- while leaving higher-level composition, animation timing, and interaction entirely to the application programmer.

The purpose of this project is to design and implement a connected sequence of six animated 2D and 3D scenes that together tell the story of a single day in a DIU student's life: waking up to an alarm, walking to a bus stop, riding a bus into the city and across a bridge toward the campus, arriving at the DIU main gate and academic building, taking an elevator up to the classroom floor, and finally sitting in a fully lit, camera-controlled 3D classroom. Each scene is implemented as an independent C++/GLUT program that reuses the same core computer-graphics principles -- primitive drawing, geometric transformations, timer-driven animation, and (in most scenes) real-time keyboard/mouse interaction and WinMM-based sound.

1.2 Motivation

The primary motivation behind this project is to combine the theoretical knowledge gained in the Computer Graphics course with a relatable, real-world narrative that a student audience can immediately recognise. Rather than building six disconnected demo scenes, the project links them into a single storyline -- a student's morning routine and commute to DIU -- which gives every technical decision (what to draw, how to animate it, and how the user interacts with it) a clear narrative purpose.

Additionally, this project provides an opportunity to:

- Apply theoretical knowledge of rasterization and transformation algorithms in a practical, multi-scene setting
- Explore real-time animation, state-machine design, and user interaction across scenes of increasing complexity
- Progress technically from simple 2D orthographic scenes toward a fully lit, camera-controlled 3D environment in the final scene
- Strengthen collaborative programming skills, since each of the six scenes was developed and integrated as a separate module by the project team

By creating a visually engaging, everyday, and culturally recognisable sequence of scenes centred on campus life at DIU, the project becomes both educational and expressive.

1.3 Objectives

1. To implement and apply fundamental line- and circle-generation principles (DDA, Bresenham's, and Midpoint Circle) consistently across all six scenes.
2. To apply 2D and 3D transformations -- translation, rotation, scaling, reflection, and shear -- to construct realistic, moving, and layered scenes.
3. To create a continuous, six-scene animated narrative using OpenGL/GLUT, moving from a 2D bedroom scene to a fully lit 3D classroom.
4. To demonstrate modular programming practice, with each visual component implemented as its own reusable draw function.
5. To demonstrate real-time rendering, timer-driven animation (glutTimerFunc at ~60 FPS), and, in four of the six scenes, synchronized WinMM sound effects.

1.4 Feasibility Study

Several existing academic projects and applications demonstrate the use of computer graphics for scene creation and character animation. Educational OpenGL/GLUT-based projects commonly include landscape rendering, animated characters, traffic and pedestrian simulations, and simple architectural walkthroughs to help students internalise graphics concepts.

Similar works reviewed during the planning of this project include:

- OpenGL-based 2D character-animation demos used in academic Computer Graphics courses (alarm-clock wake-up sequences, walking-cycle demonstrations)
- Traffic and street-scene simulations that combine moving vehicles, pedestrians, and traffic signals using primitive-based 2D rendering
- Simple 3D room/interior walkthroughs built with glutSolidCube-style primitives and basic Phong lighting to demonstrate the transition from 2D to 3D graphics pipelines

These reference works confirm that using OpenGL together with classical rasterization principles and standard 2D/3D transformations is both practical and effective for building an animated, multi-scene project such as this one. The methodologies observed in these systems emphasize modular design, efficient timer-based rendering, and real-time state updates -- all of which are directly reflected in the architecture of this project.

This project follows a similar technical approach but distinguishes itself by chaining six scenes into one continuous storyline, and by deliberately escalating the graphics complexity from a static 2D bedroom in Scene 1 to a dynamic, lit, camera-controlled 3D classroom in Scene 6.

1.5 Gap Analysis

Although many existing student graphics projects demonstrate individual rasterization algorithms or a single animated scene, most of them stop at one isolated demo and rarely connect multiple

scenes into a coherent story, and very few progress from 2D to 3D rendering within the same project.

The identified gaps in typical, generic Computer Graphics lab projects are:

- A single, isolated scene rather than a connected multi-scene narrative
- Limited or no state-machine logic to transition an animation between distinct phases (e.g., sleeping to standing, doors closing to ascending)
- Rare use of synchronized sound effects alongside visual animation
- Almost no projects that bridge 2D orthographic rendering with a genuinely lit, camera-controlled 3D scene in the same body of work

This project addresses these gaps by:

- Chaining six independent scenes into a single storyline representing a DIU student's day
- Implementing explicit timeline and finite-state-machine logic in every scene (e.g., ALARM_TIME/WAKE_END/WALK_START stage boundaries in Scene 1, and a door-close/ascend/floor-update/door-open cycle in Scene 5)
- Layering WinMM PlaySoundA-based ambient and event sound effects into four of the six scenes
- Escalating technical scope across the sequence, culminating in a fully lit, camera-controllable 3D classroom (Scene 6) built from cube, cylinder, and sphere primitives under Phong-style lighting

1.6 Project Outcome

The final output of this work is a set of six functional, real-time-rendered OpenGL/GLUT applications that, played in order, depict a complete narrative arc:

- Scene 1 -- Morning Alarm in a Celebrity's Room: The narrative opens inside a bedroom at dawn. A sleeping character is woken by a ringing alarm clock, sits up, stands, and walks toward a window as the sun rises outside.
- Scene 2 -- Student Goes to the Bus Stop: The scene follows the same student stepping out of the house onto a city street lined with buildings, trees, and traffic, walking toward a bus stop as a bus, car, and rickshaw pass by.
- Scene 3 -- DIU Student Bus Journey: The student boards the bus, and the scene depicts the journey itself — a moving city panorama with parallax buildings, a river and bridge crossing, and passengers visible through the bus windows — ending as the bus approaches the DIU campus.
- Scene 4 -- DIU Campus Arrival: The bus journey ends as the student arrives at the Daffodil International University campus, walking past the main gate, academic building, prayer facility, and football field toward class.
- Scene 5 -- Elevator Ride to the Classroom Floor: Inside the academic building, the student enters a lobby, boards an elevator, the doors close, and the cabin ascends while a floor-number display counts upward, ending with the doors reopening on the classroom floor.

- Scene 6 -- 3D DIU Classroom: The final scene transitions from 2D to a lit, camera-controlled 3D classroom populated with desks, chairs, a whiteboard, a projector, and seated students/teacher, closing the student's daily journey inside the lecture hall.

Key overall features of the finished project include:

- A consistent set of drawing primitives (rectangles, circles/ellipses, lines, polygons, and, in Scene 6, cuboids/cylinders/spheres) reused and re-composed across all six scenes
- Timer-driven real-time animation running at approximately 60 FPS (16 ms glutTimerFunc interval) in every scene
- Keyboard- and, in several scenes, mouse-driven interactivity that lets the user pause, restart, jump between animation stages, or manually control a character
- Synchronized WinMM ambient and event sound effects in Scenes 2, 3, 4, and 6
- A deliberate technical progression from a static 2D orthographic bedroom scene to a dynamic, lit, camera-controlled 3D classroom, demonstrating a broad command of both 2D and 3D computer graphics fundamentals.

Chapter 2

System Architecture

2.1 Requirement Analysis & Design Specification

2.1.1 Scene Overview

The project is organized as six sequential scenes, each implemented as an independent GLUT application (folders S01-S06 in the accompanying source archive), that together represent a single day in the life of a DIU student -- from waking up at home to sitting in a lecture inside a fully rendered 3D classroom.

Scene 1: Morning Alarm in a Celebrity's Room

The narrative opens inside a bedroom at dawn. A sleeping character is woken by a ringing alarm clock, sits up, stands, and walks toward a window as the sun rises outside.

Principal graphical components: Bed and pillow (rectangles and ellipses), a rotating ceiling fan (four blades via rotated rectangles about a pivot), a wall-mounted alarm clock (concentric circles with rotating hands), a bookshelf (stacked rectangles), a framed wall picture, and a window frame that reveals an outdoor sunrise composed of a gradient sky, a rising sun, drifting clouds, distant hills, and flapping birds. The character itself is built from ellipses (head, torso) and line segments (limbs) blended smoothly between sleeping, sitting, and standing poses.

Scene 2: Student Goes to the Bus Stop

The scene follows the same student stepping out of the house onto a city street lined with buildings, trees, and traffic, walking toward a bus stop as a bus, car, and rickshaw pass by.

Principal graphical components: A row of multi-storey buildings with window grids, a two-lane road with lane markings, roadside trees built from a rectangular trunk and layered circular canopies, a traffic signal with cycling red/yellow/green lights, a street lamp, a public bus and a rickshaw rendered from rectangles, circles (wheels) and trapezoidal panels, and the walking student figure with a swinging-arm/leg cycle.

Scene 3: DIU Student Bus Journey

The student boards the bus, and the scene depicts the journey itself — a moving city panorama with parallax buildings, a river and bridge crossing, and passengers visible through the bus windows — ending as the bus approaches the DIU campus.

Principal graphical components: A layered city skyline, a river rendered as a gradient-filled band beneath a girder bridge, roadside lamps and trees that scroll past to create a parallax motion effect, and the bus itself with rotating wheels and passenger silhouettes visible through window cut-outs.

Scene 4: DIU Campus Arrival

The bus journey ends as the student arrives at the Daffodil International University campus, walking past the main gate, academic building, prayer facility, and football field toward class.

Principal graphical components: A stylized university main gate, a multi-storey academic building with a repeating window grid, a smaller prayer facility, a marked football field complete with goal frames and penalty box lines, an intersection where the campus road meets the main avenue, and a small crowd of independently animated walking students represented with the same limb-articulated figure used in earlier scenes.

Scene 5: Elevator Ride to the Classroom Floor

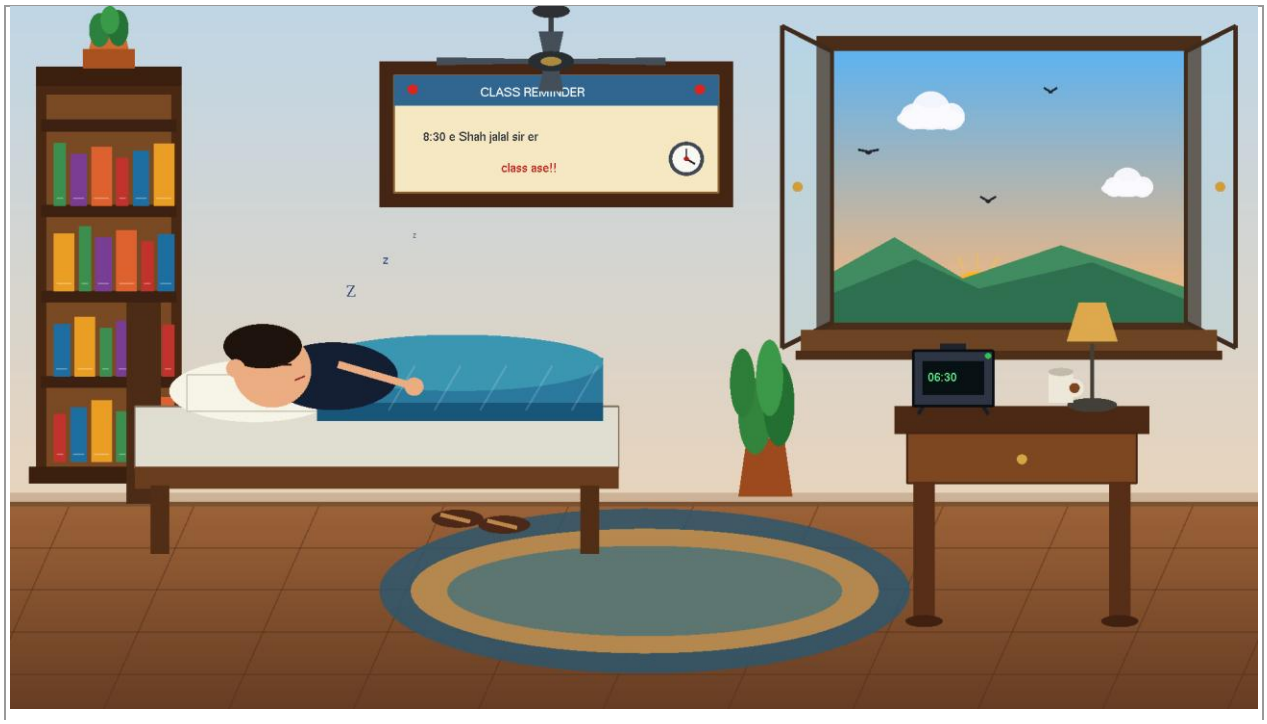
Inside the academic building, the student enters a lobby, boards an elevator, the doors close, and the cabin ascends while a floor-number display counts upward, ending with the doors reopening on the classroom floor.

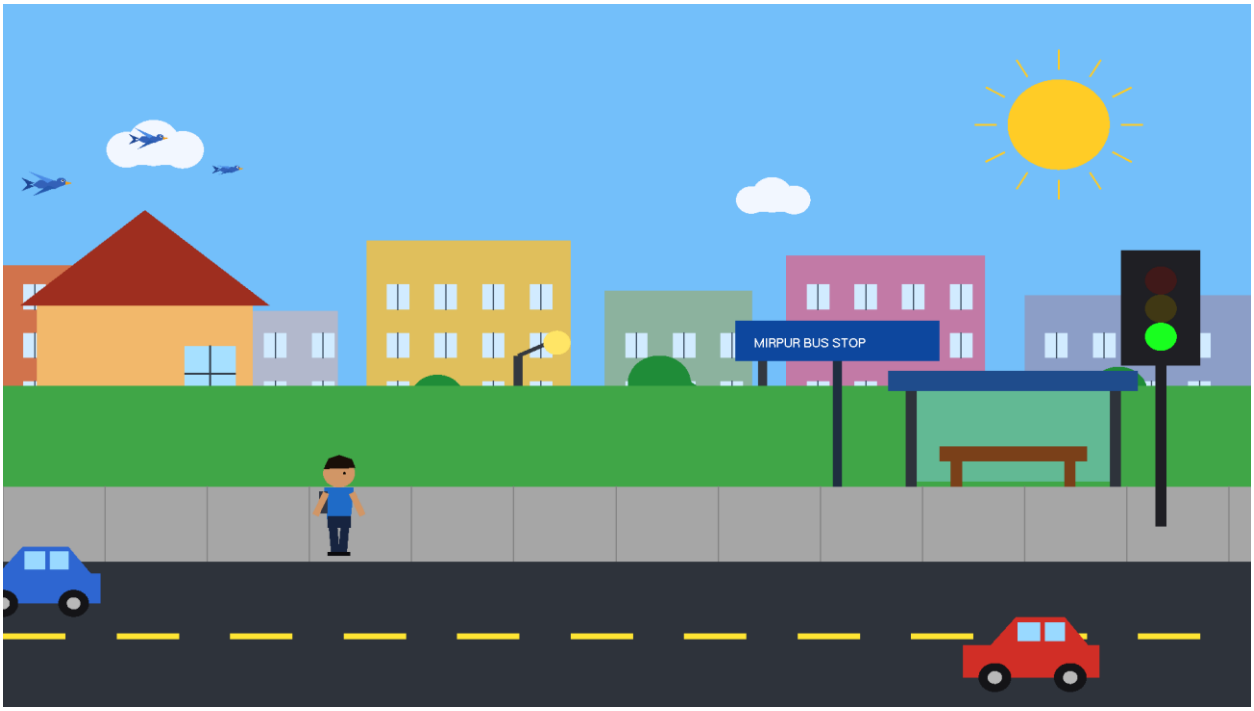
Principal graphical components: A lobby with a gradient-shaded back wall, an elevator frame and a pair of sliding doors that translate apart and together, an interior cabin panel, a boy character who enters and stands inside the cabin, an additional passenger silhouette for scene realism, and an on-screen floor-number / direction-arrow HUD text element.

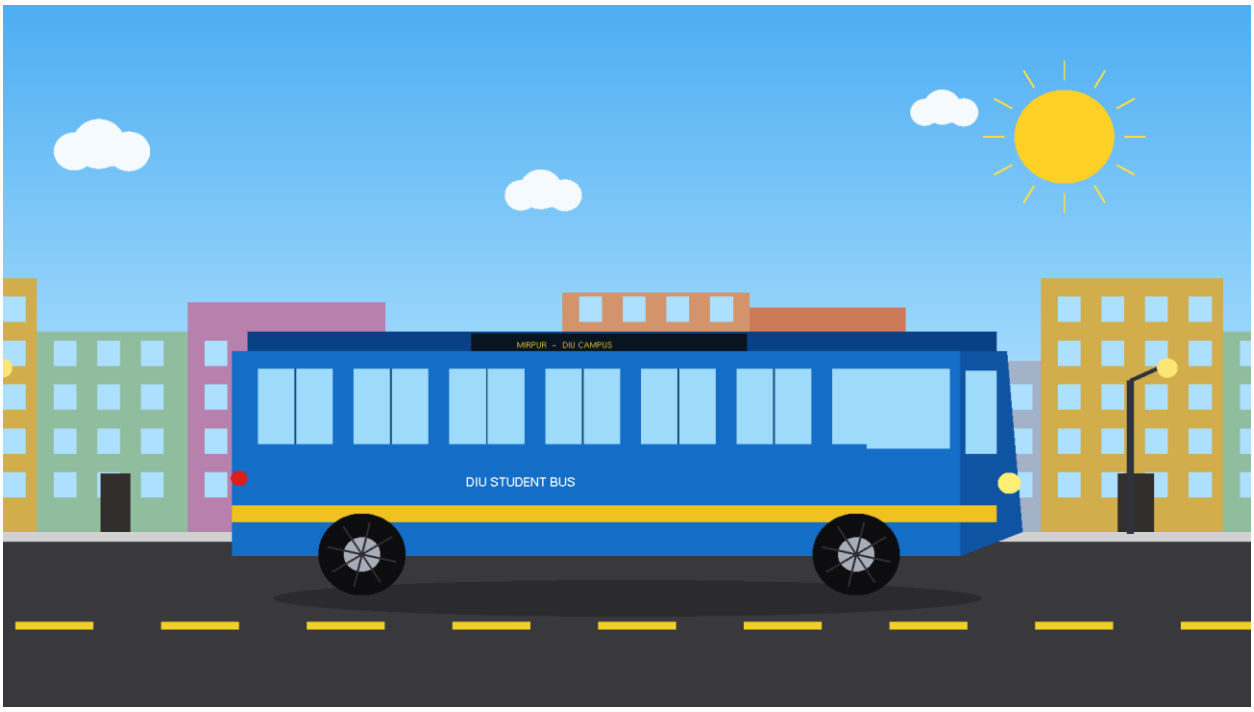
Scene 6: 3D DIU Classroom

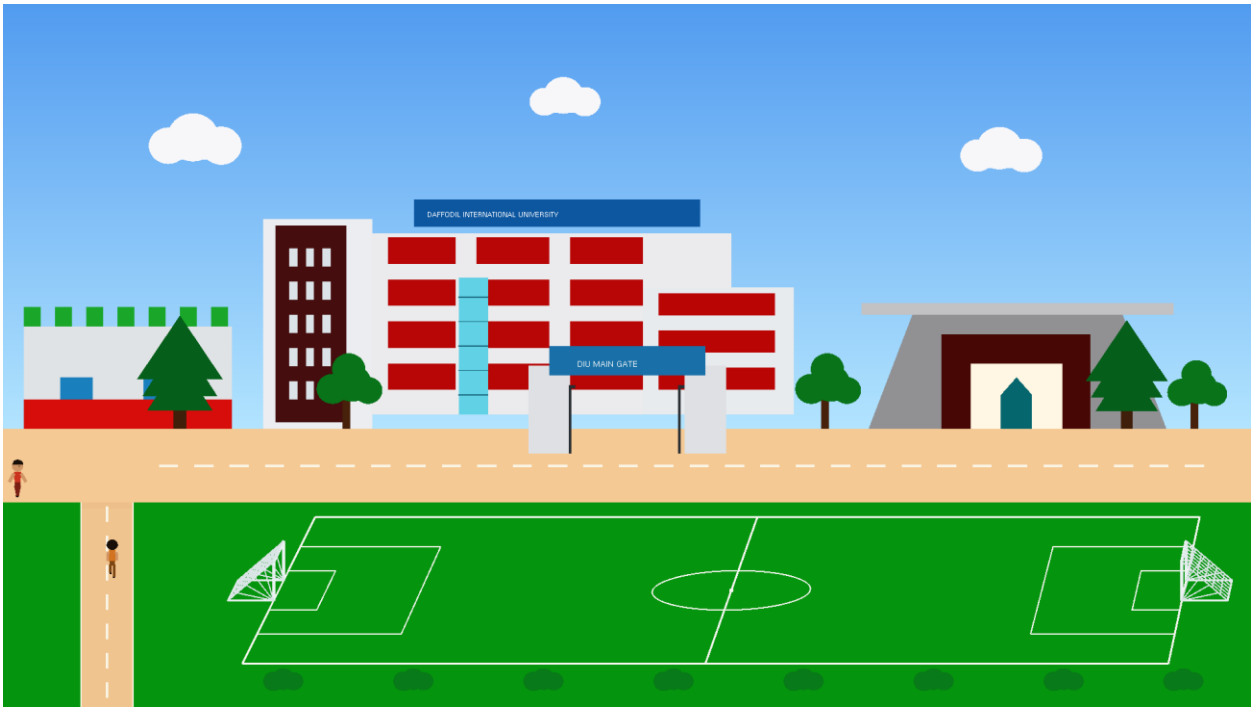
The final scene transitions from 2D to a lit, camera-controlled 3D classroom populated with desks, chairs, a whiteboard, a projector, and seated students/teacher, closing the student's daily journey inside the lecture hall.

Principal graphical components: A fully enclosed classroom volume (walls, floor, ceiling) with windows and a door, a whiteboard and teacher's table at the front, rows of student desks and chairs, an overhead projector, a rotating ceiling fan, an air-conditioning unit, tube lights, a wall clock, laptops on desks, and articulated human figures (teacher and seated students) all built from primitive 3D solids (glutSolidCube-based cube(), a custom cylinder(), and a custom sphere()) under Phong-style lighting (glEnable(GL_LIGHTING), glLightfv, glMaterialfv).




MIRPUR BUS STOP


MIRPUR – DIU CAMPUS
DIU STUDENT BUS


DAFFODIL INTERNATIONAL UNIVERSITY
DIU MAIN GATE

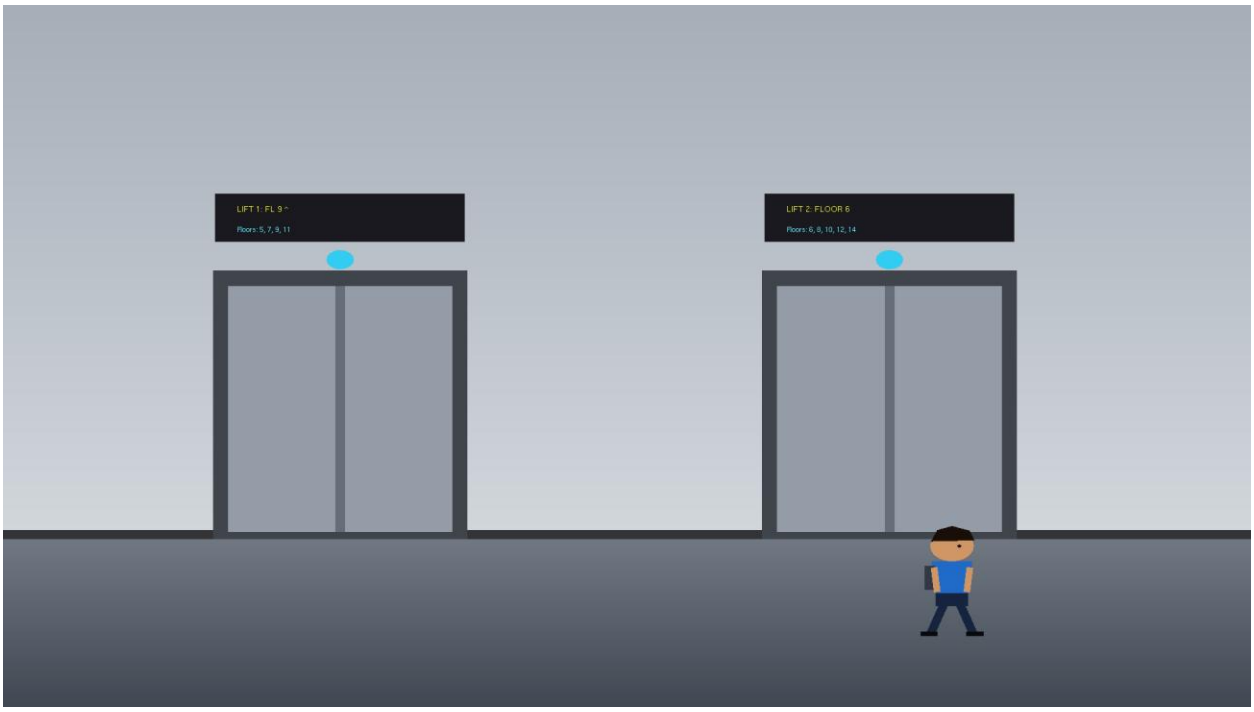

LIFT 1: FL 9 ^
Floors: 5, 7, 9, 11
LIFT 2: FLOOR 6
Floors: 6, 8, 10, 12, 14



Figure 01: Project Flowchart (Scene 1 -> Scene 2 -> Scene 3 -> Scene 4 -> Scene 5 -> Scene 6)

2.1.2 Object Components

Across the six scenes, every visual object is built from a small, reusable set of geometric primitives:

- Rectangles / Quads: used for room walls, buildings, roads, vehicle bodies, elevator doors, desks, and classroom walls and floor.
- Circles / Ellipses: used for the sun, character heads, clock faces, wheels, cloud lobes, and tree canopies, generated by the project's shared circle()/ellipse() routine.
- Lines: used for character limbs, fan blades, road/field markings, window-frame edges, and lamp posts.
- Filled polygons: used for hills, panels, and other irregular multi-vertex shapes such as the window's angled casement panels in Scene 1.
- 3D solids (Scene 6 only): cuboids (cube()), cylinders (cylinder()), and spheres (sphere()), combined with OpenGL lighting and material calls (glEnable(GL_LIGHTING), glLightfv, glMaterialfv) to render a genuinely three-dimensional, shaded classroom.

2.2 Use of Modern Tools

This section highlights the modern software tools and technologies employed in the project to enhance visualization, animation control, and rendering performance.

- OpenGL / FreeGLUT -- the primary graphics library used for hardware-accelerated rendering, primitive drawing, and (in Scene 6) lighting and camera transformation handling.

- Code::Blocks -- used as the Integrated Development Environment (IDE) for writing, compiling, and debugging the C++ source code, as reflected in the accompanying .cbp project files for each scene.
- GLUT windowing/timer system -- managed the window, handled keyboard and mouse callbacks (glutKeyboardFunc, glutSpecialFunc, glutMouseFunc), and controlled ~60 FPS animation timing through glutTimerFunc with a 16-millisecond interval in every scene.
- WinMM (Windows Multimedia API) -- linked via #pragma comment(lib, "winmm.lib") and used through PlaySoundA in Scenes 2, 3, 4, and 6 to play looped ambient audio (traffic, bus engine, wind, classroom ambience, fan) and one-shot event sounds.
- C++ Programming Language -- the core language used for implementing the shared drawing utilities, timeline/state-machine logic, and modular scene-specific functions.

2.3 Algorithms Used

To meet the academic requirements of a Computer Graphics project, the classical line- and circle-generation algorithms taught in the course underpin every primitive drawn on screen, even though the project's shared helper functions (line(), circle(), rectangle(), filledPolygon()) expose them through a convenient OpenGL-based interface rather than a manual pixel-plotting loop:

6. DDA (Digital Differential Analyzer) Line Algorithm: Conceptually applied wherever a set of points must be generated at a constant unit increment along one axis -- for example the evenly spaced dashed lane markings on the roads in Scenes 2-4, the sun's radiating rays in Scene 1, and the constant-rate scrolling of parallax trees and lamp posts in Scene 3. The DDA approach increments coordinates by a unit step along the major axis and computes the corresponding step in the other axis from the line's slope, which suits these repetitive, evenly spaced elements well.
7. Bresenham's Line Algorithm: Used conceptually for structural, well-defined straight edges such as room and building outlines, window frames, elevator door edges, football-field boundary lines, and classroom furniture edges. As an integer-based, decision-parameter-driven algorithm, Bresenham's method avoids floating-point round-off and produces clean, consistent rasterization, which is why it underlies every axis-aligned or diagonal structural edge across the six scenes.
8. Midpoint Circle Algorithm: Underlies every circular or elliptical component -- the sun, character heads, alarm clock and wall clock faces, vehicle wheels, and (extended into 3D) the sphere-based human heads in Scene 6. The Midpoint Circle method uses a decision variable and eight-way symmetry to compute one octant of the circle and mirror it into the remaining seven, avoiding expensive trigonometric evaluation for every point.

Scene-specific notes on where each principle is most visibly applied are summarized below:

- Morning Alarm in a Celebrity's Room: The circular alarm clock face and the character's head are generated with the project's midpoint-circle-based circle() routine, while the fan blades and window-frame edges rely on Bresenham-style straight-line rasterization.
- Student Goes to the Bus Stop: Traffic-signal poles, road lane dashes, and building window grids are produced with Bresenham-style line rasterization for crisp axis-aligned and diagonal edges, while the vehicle wheels and the student's head use the midpoint-circle routine; the

DDA approach underlies the evenly spaced dashed lane markings that must increment at a constant unit rate along the road.

- DIU Student Bus Journey: The bridge girders and roadside lamp posts are drawn using Bresenham's algorithm for their straight structural edges; the bus wheels and passenger heads apply the midpoint-circle routine; the DDA approach governs the constant-rate horizontal scrolling of the parallax trees and lamps that must move in fixed unit steps per frame to remain synchronized.
- DIU Campus Arrival: The football-field boundary lines, goal frames, and penalty box are drawn with Bresenham's algorithm to guarantee straight, evenly weighted lines; the round and pine tree canopies rely on the midpoint-circle routine; the DDA approach is used for the evenly dashed centre line of the campus road and the building's window grid, both of which require constant unit increments along one axis.
- Elevator Ride to the Classroom Floor: The elevator door edges and frame are produced with Bresenham's algorithm for clean vertical and horizontal segments; the character heads use the midpoint-circle routine; the constant-rate closing/opening motion of the doors and the steadily incrementing floor counter both follow the DDA-style principle of equal unit increments per timer tick.
- 3D DIU Classroom: Because this scene is rendered in true 3D, straight-edged furniture (desks, chairs, door and window frames, the whiteboard) is generated from cuboid primitives whose edges correspond to Bresenham-rasterized lines at the driver level, while rounded components — the classroom clock face, fan hub, and human heads — are built from the circle/sphere routines that extend the midpoint-circle principle into three dimensions.

2.4 2D Transformations Applied

Geometric transformations are the mechanism by which every static primitive becomes part of a moving, animated scene. All six scenes rely on OpenGL's model-view transformation stack (glTranslatef, glRotatef, glScalef, wrapped in glPushMatrix()/glPopMatrix() pairs) in addition to direct coordinate arithmetic for simpler cases.

Translation

Used to move objects across the scene for example, the character walking across the bedroom and later the campus road, vehicles and pedestrians moving horizontally through Scenes 2-4, the elevator doors sliding open and shut, and each seated figure being positioned within the 3D classroom grid in Scene 6.

Rotation

Used to simulate mechanical and orientation-based movement the ceiling fan blades in Scenes 1 and 6, the alarm and wall clock hands, the bus and vehicle wheels in Scenes 2-3, and the 3D camera orientation set up via setupCamera() in Scene 6.

Scaling

Used to vary object size for depth cues and pose changes background buildings and trees are scaled smaller to suggest distance in Scenes 2-4, the sit-to-stand pose transition in Scene 1 briefly scales

the character vertically (glScalef(1.0f, 0.80f + 0.20f * p, 1.0f)), and near/far parallax layers are scaled differently in Scene 3.

Reflection

Applied where mirrored geometry is needed for example the left/right symmetry of the elevator doors in Scene 5 (each half is the horizontal mirror of the other about the cabin's centre line) and the paired left/right window casement panels in Scene 1.

Shear

Used sparingly for stylized geometry the angled window casement panels in Scene 1 and the trapezoidal vehicle-body panels in Scenes 2-3 are constructed as sheared quadrilaterals via filledPolygon() rather than pure axis-aligned rectangles, giving the illusion of perspective within an orthographic 2D projection.

2.5 Animation Logic

Animation in every scene is driven by a Global Timer Loop registered with glutTimerFunc(16, timer, 0), which re-invokes itself every frame (approximately 60 FPS) and calls glutPostRedisplay() to trigger a fresh render.

9. State Variables: Each scene maintains a small set of state variables -- for example sceneTime, fanAngle, and manualCharacterX in Scene 1, or the floor counter and door-offset variables in Scene 5 -- that are updated once per timer tick and read back during display() to position and orient every object.
10. Increment Logic: Continuous motion is produced with simple additive updates each frame, for example fanAngle += <rate> to spin the ceiling fan, or a smooth01()-eased interpolation (t*t*(3-2t)) used to ease the sit-to-stand transition and the sunrise motion in Scene 1.
11. Timeline / Finite-State-Machine Logic: Rather than a single continuous loop, each scene defines named stage boundaries -- ALARM_TIME, WAKE_END, SIT_END, WALK_START, WALK_END, and ALARM_OFF_TIME in Scene 1, or the door-close / ascend / floor-update / door-open cycle inside updateSimulation() in Scene 5 -- so that sceneTime being compared against these constants determines which pose or phase is currently rendered.
12. Cyclic / Loop Reset: To keep each demo replayable, an autoLoop flag resets sceneTime back to zero once END_TIME is reached (resetScene()), and looping background elements (parallax trees, lamps, birds in Scenes 2-4) wrap around using std::fmod so they re-enter the visible frame smoothly rather than disappearing abruptly.
13. Trigonometric Motion: For natural, organic movement, sine and cosine are used throughout -- cloud drifting (sin(sceneTime * 0.25f)), bird bobbing and wing-flap phase, and the smooth easing curves applied to poses and sunrise motion.

Chapter 3

Implementation and Results

This chapter explains the step-by-step implementation process of the Computer Graphics project, including the development of objects, application of algorithms, and integration of animations across all six scenes. It also presents the key features, transformations, and visual outputs achieved through OpenGL rendering, and requests dedicated screenshots for the start, mid, and end phase of every scene.

3.1 Implementation

The project was implemented in C++ with OpenGL and GLUT, following a modular programming approach in every scene. Each visual object whether a piece of furniture, a vehicle, a character, or a building was created using its own dedicated draw function to ensure clarity, reusability, and ease of debugging, and each scene's source lives in its own project folder (S01 through S06) with a matching Code::Blocks project file.

Scene Construction & Layering

Every scene is constructed back-to-front: a background layer (sky, walls, or gradient) is drawn first, followed by mid-ground structural elements (buildings, furniture, roads), and finally foreground animated elements (characters, vehicles, doors) so that later draw calls correctly occlude earlier ones under OpenGL's default painter's-algorithm-style 2D compositing. In Scene 6, this layering is extended into true 3D using OpenGL's depth buffer together with glEnable(GL_LIGHTING) so that solid objects occlude one another correctly regardless of draw order.

Implementation Details of the Core Rasterization Principles

The project successfully integrates the three fundamental rasterization principles required for the course across all scenes:

- Bresenham's Line Algorithm underlies structural straight edges: room/building outlines, window and elevator-door frames, football-field boundary and goal lines, road lane edges, and classroom furniture edges in Scene 6.
- DDA Line Algorithm underlies evenly, constant-rate-spaced elements: dashed road lane markings, the sun's radiating rays, and the constant-step scrolling of parallax trees and lamp posts.
- Midpoint Circle Algorithm underlies every circular element: the sun, clock faces, character heads, vehicle wheels, and (extended to 3D) the sphere-based heads of the human figures in Scene 6.

These principles are exposed through the project's own shared helper functions (line(), circle()/drawCircle(), rectangle()/drawRectangle(), filledPolygon()) so that every one of the dozens of scene-specific draw functions listed below can call a single, consistent primitive layer rather than re-implementing rasterization logic per scene.

Object Design and Modular Functions

Each scene decomposes its visual content into a list of modular, reusable draw functions. The complete function inventory extracted from the source code is as follows:

Scene 1 Morning Alarm in a Celebrity's Room (source: S01/main.cpp)

Key functions: drawRoomShell(), drawBed(), drawTable(), drawAlarmClock(), drawFan(), drawBookshelf(), drawWallPicture(), drawWindowAndOutside(), drawSleepingCelebrity(), drawSeatedCelebrity(), drawStandingCelebrity(), drawSun(), drawCloud(), drawBird(), drawHUD().

Scene 2 Student Goes to the Bus Stop (source: S02/main.cpp)

Key functions: drawSky(), drawBuildings(), drawGroundAndRoad(), drawTrees(), drawHouse(), drawStreetLight(), drawTrafficSignal(), drawBusStop(), drawBus(), drawCar(), drawRickshaw(), drawStudent(), drawClouds(), drawBirds(), drawInterface().

Scene 3 DIU Student Bus Journey (source: S03/main.cpp)

Key functions: drawSky(), drawCityBackground(), drawDIUCampus(), drawRiverAndBridge(), drawNormalRoad(), drawMovingTrees(), drawMovingLamps(), drawBus(), drawWheel(), drawPassengers(), drawClouds(), drawBirds(), drawLamp(), updateStory(), drawInterface().

Scene 4 DIU Campus Arrival (source: S04/main.cpp)

Key functions: drawSky(), drawCampusRoad(), drawMainGate(), drawMainAcademicBuilding(), drawLeftFacility(), drawPrayerBuilding(), drawFootballField(), drawGoalFrame(), drawPitchBox(), drawIntersection(), drawPineTree(), drawRoundTree(), drawBush(), drawWalkingStudents(), drawStudent(), drawClouds().

Scene 5 Elevator Ride to the Classroom Floor (source: S05/main.cpp)

Key functions: drawLobbyBackground(), drawElevatorInteriorAndFrame(), drawElevatorDoors(), drawBoy(), drawPassenger(), drawGradientRectV(), drawText(), updateSimulation().

Scene 6 3D DIU Classroom (source: S06/main.cpp)

Key functions: setupCamera(), setupLighting(), setMaterial(), drawWalls(), drawFloor(), drawWindows(), drawDoor(), drawBoard(), drawTeacherTable(), drawDesk(), drawChair(), drawClassroomFurniture(), drawProjector(), drawFan(), drawAC(), drawTubeLight(), drawClock(), drawLaptop(), drawHuman(), drawPeople(), cube(), cylinder(), sphere().

This modular structure improves code readability, maintainability, and debugging efficiency -- a change to how a wheel is drawn, for instance, automatically applies everywhere that draw function is called within its scene.

3.2 Output & Screenshots

The final output of the project is a set of six real-time animated scenes depicting the continuous story of a DIU student's day. For each of the six scenes, three screenshots are required below, captured at the Start, Mid, and End phase of that scene's animation timeline, to visually document how the scene evolves from its opening state through its climax to its resolution.

Scene 1: Morning Alarm in a Room

Figure 02a-c below document the three required phases of this scene.

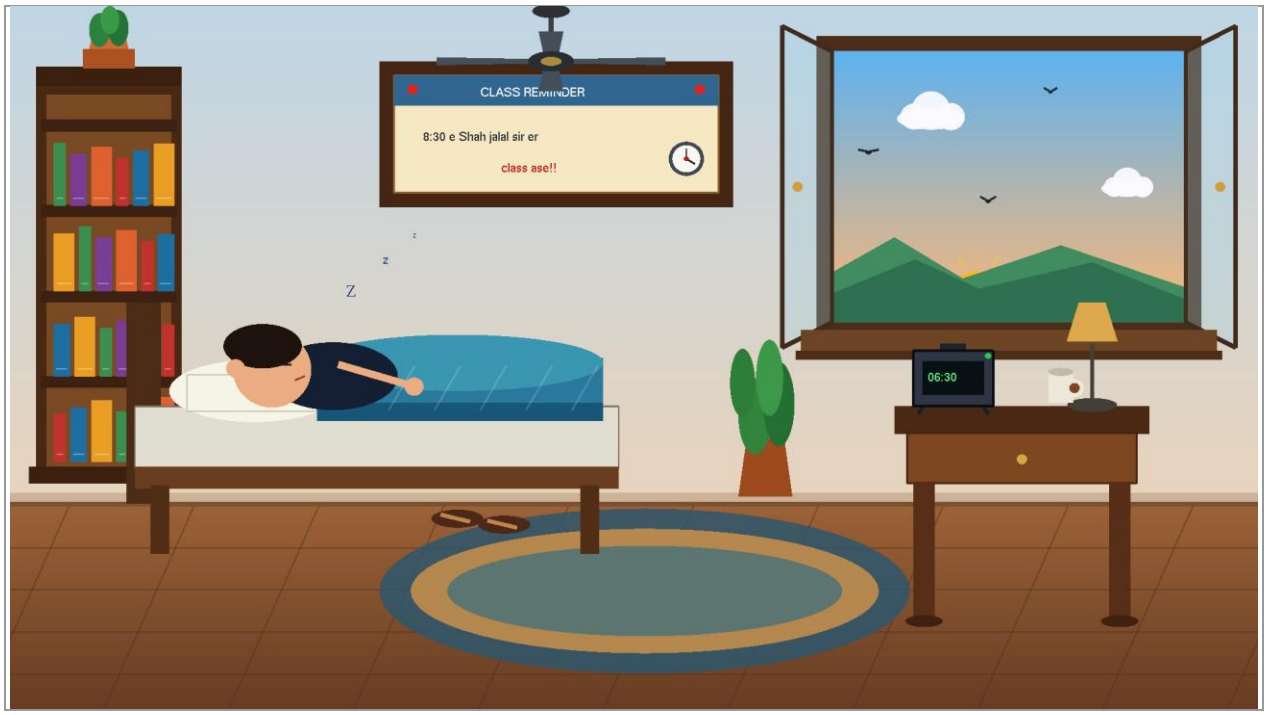


Figure 02a: Scene 1 - Start Phase

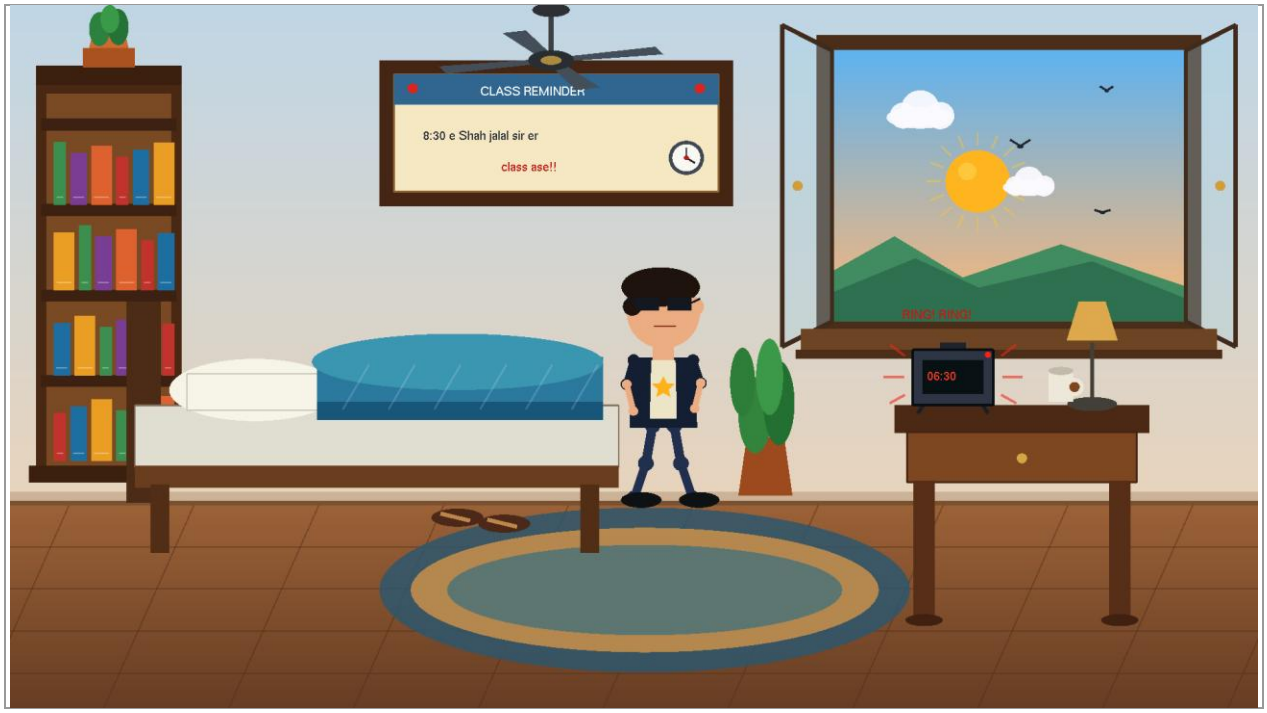


Figure 02b: Scene 1 - Mid Phase

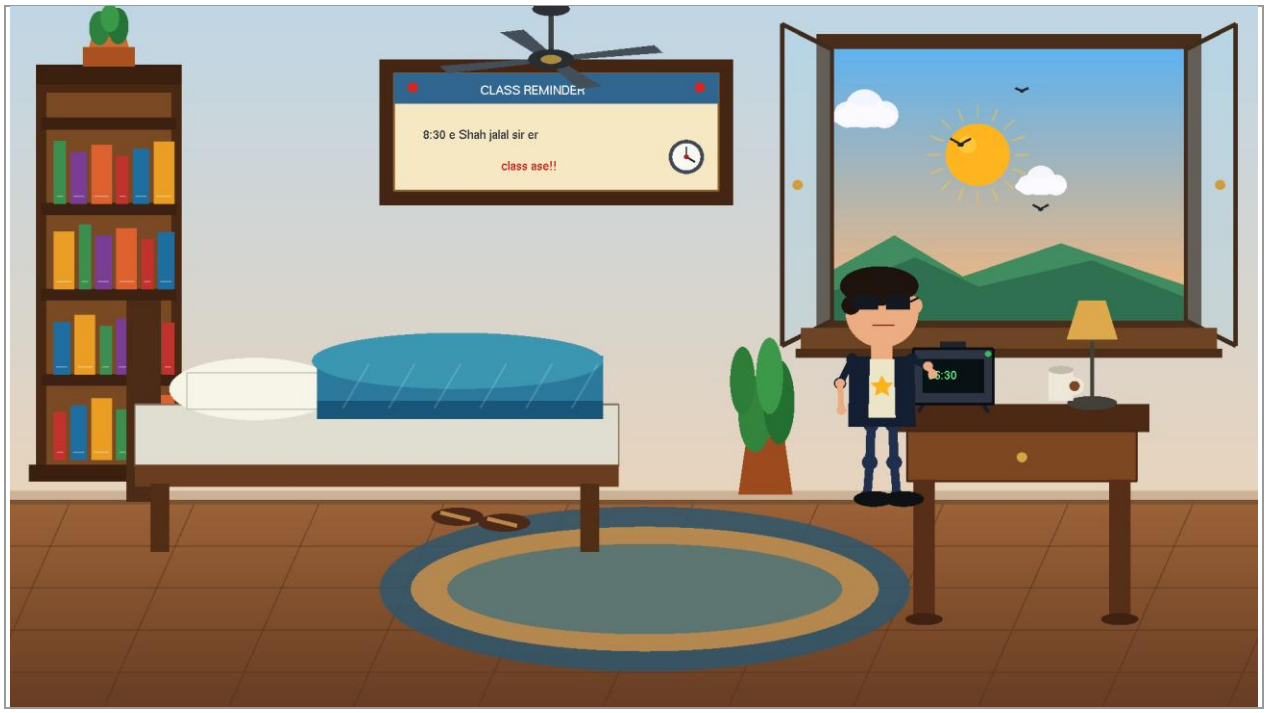


Figure 02c: Scene 1 - End Phase

Scene 2: Student Goes to the Bus Stop

Figure 03a-c below document the three required phases of this scene.

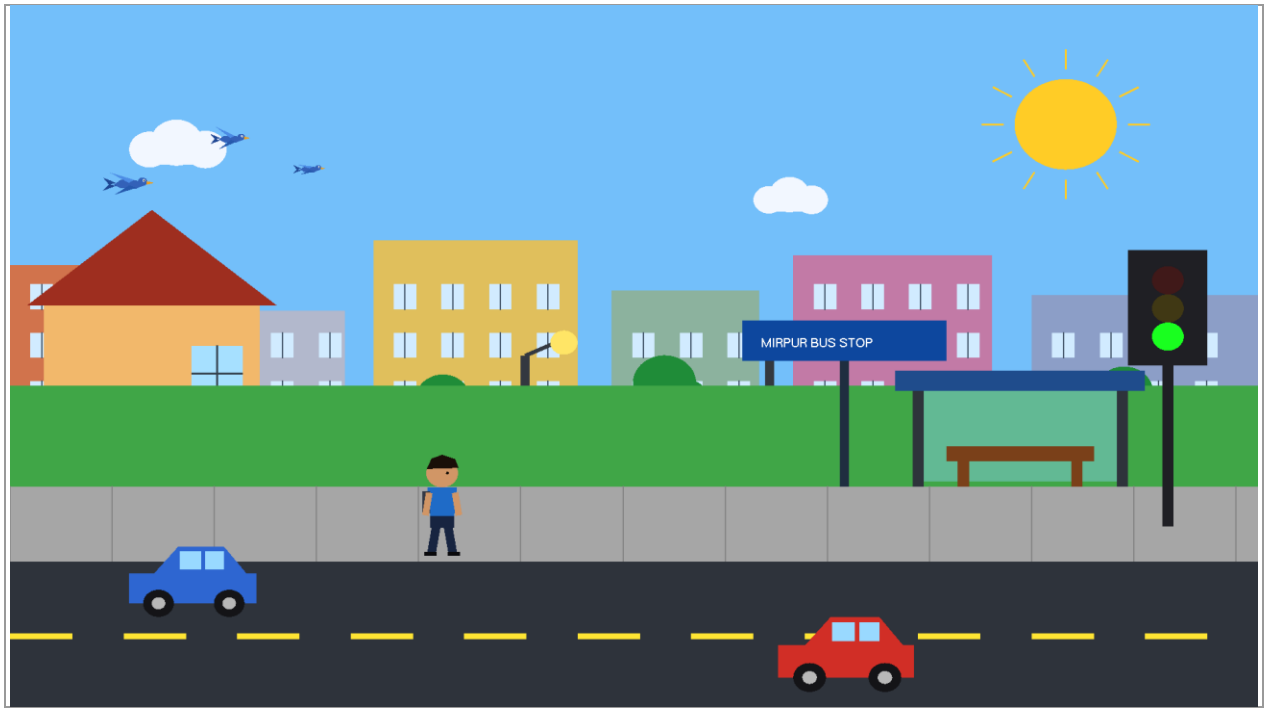


Figure 03c: Scene 1 - Start Phase

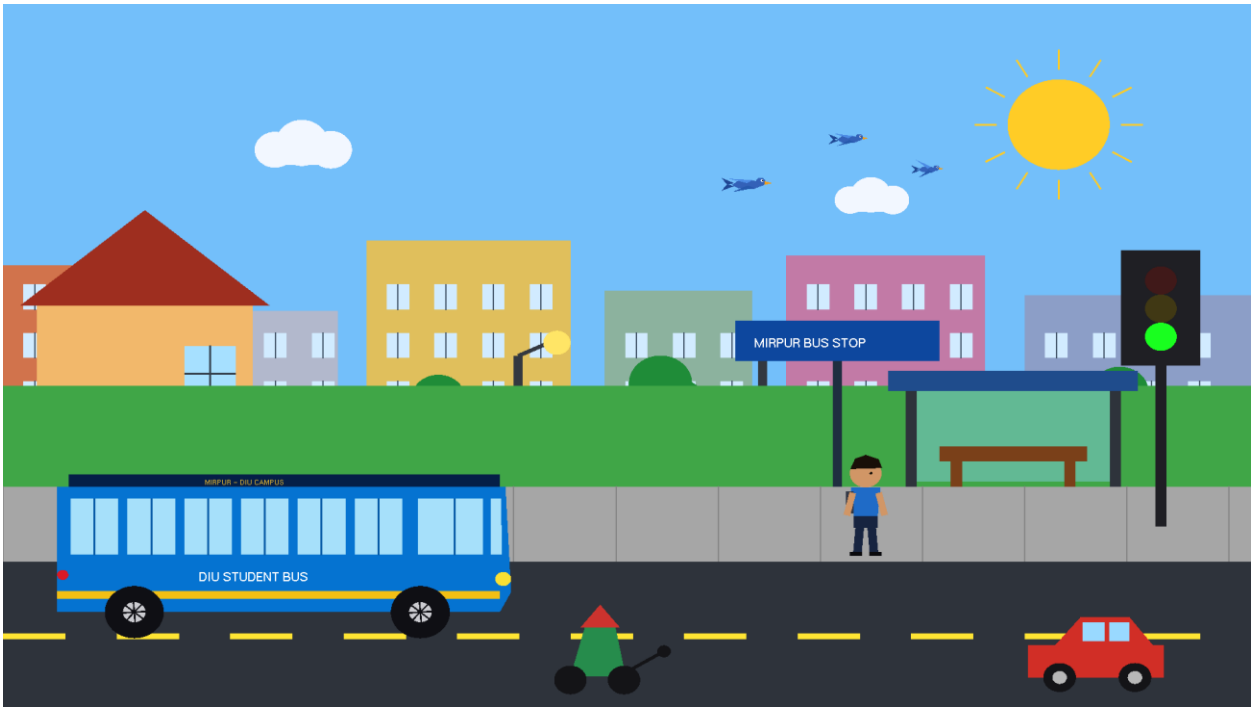


Figure 03b: Scene 2 - Mid Phase



Figure 03c: Scene 2 - End Phase

Scene 3: DIU Student Bus Journey

Figure 04a-c below document the three required phases of this scene.

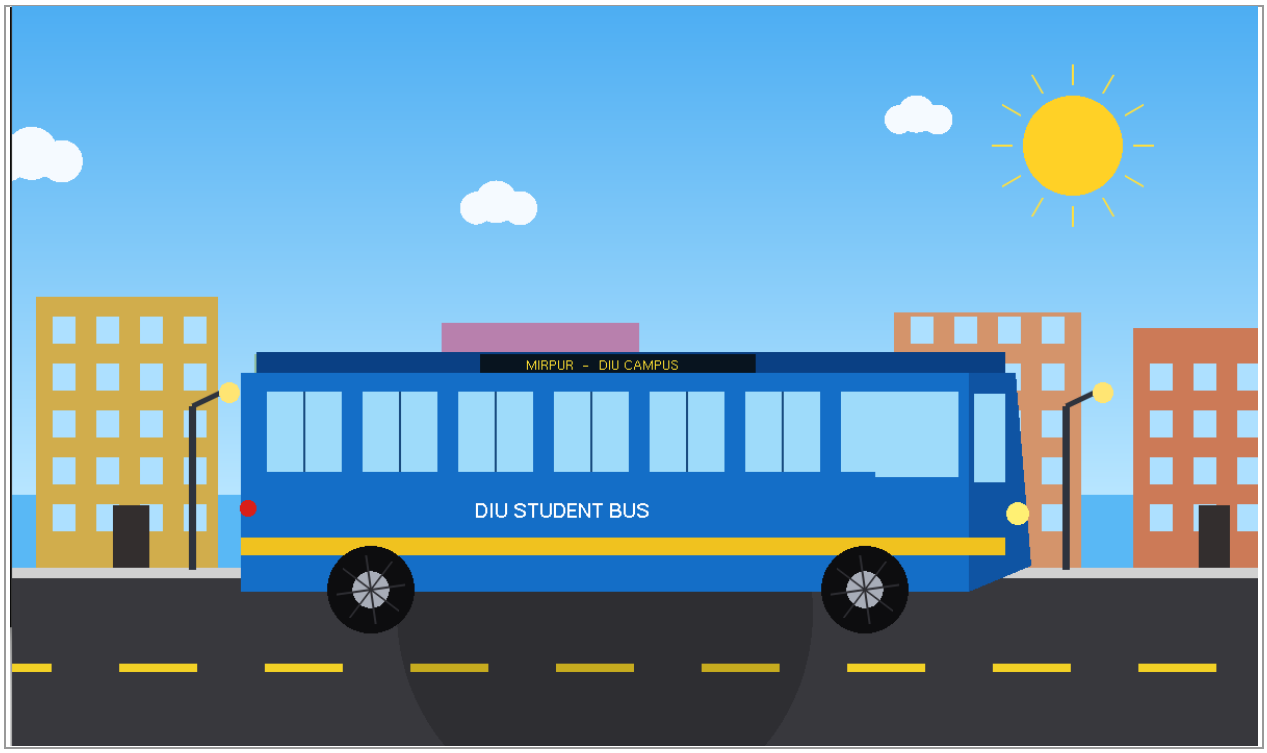


Figure 04a: Scene 3 - Start Phase

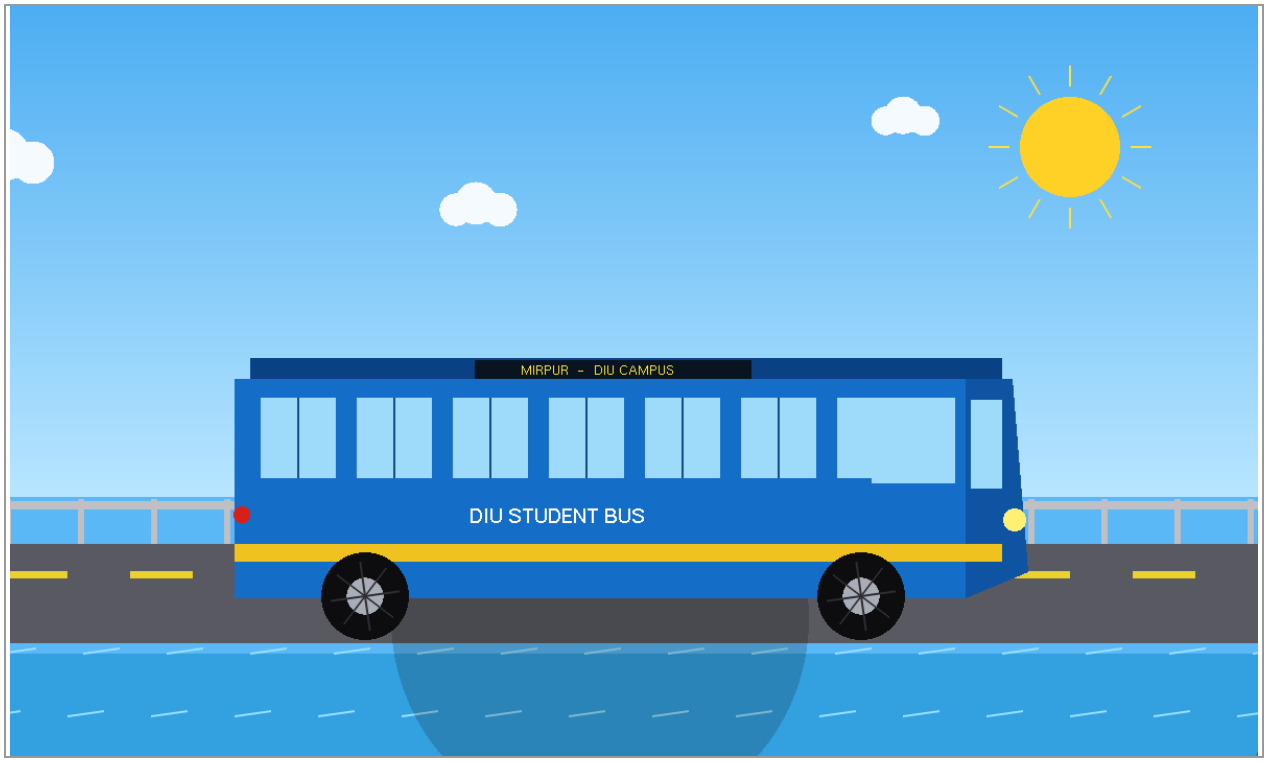


Figure 04b: Scene 3 - Mid Phase

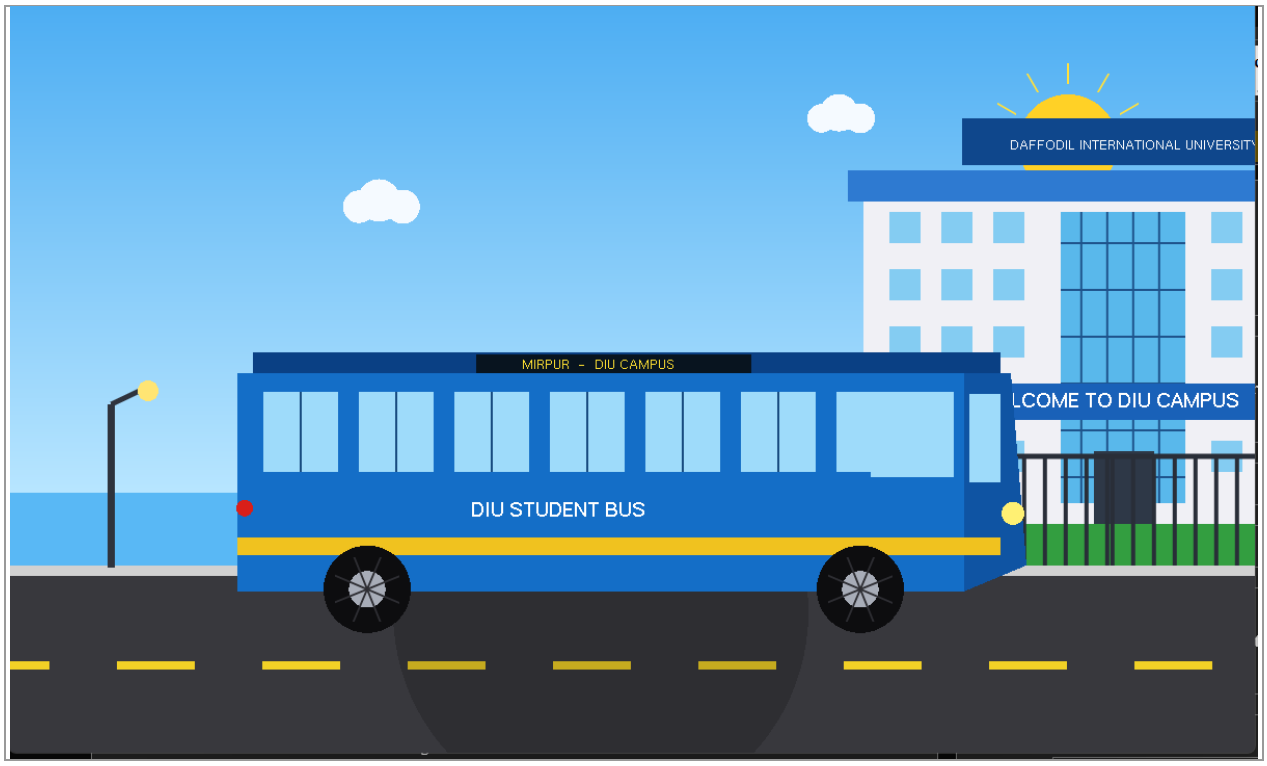


Figure 04c: Scene 3 - End Phase

Scene 4: DIU Campus Arrival

Figure 05a-c below document the three required phases of this scene.

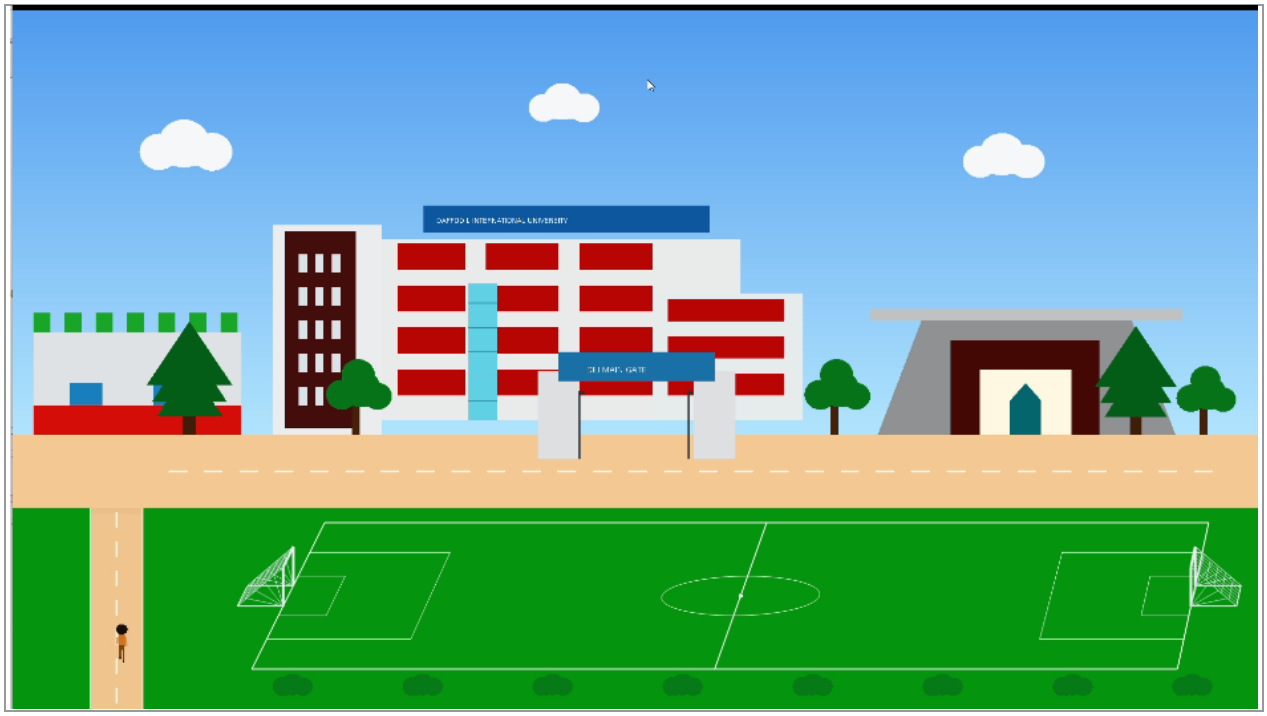


Figure 05a: Scene 4 - Start Phase

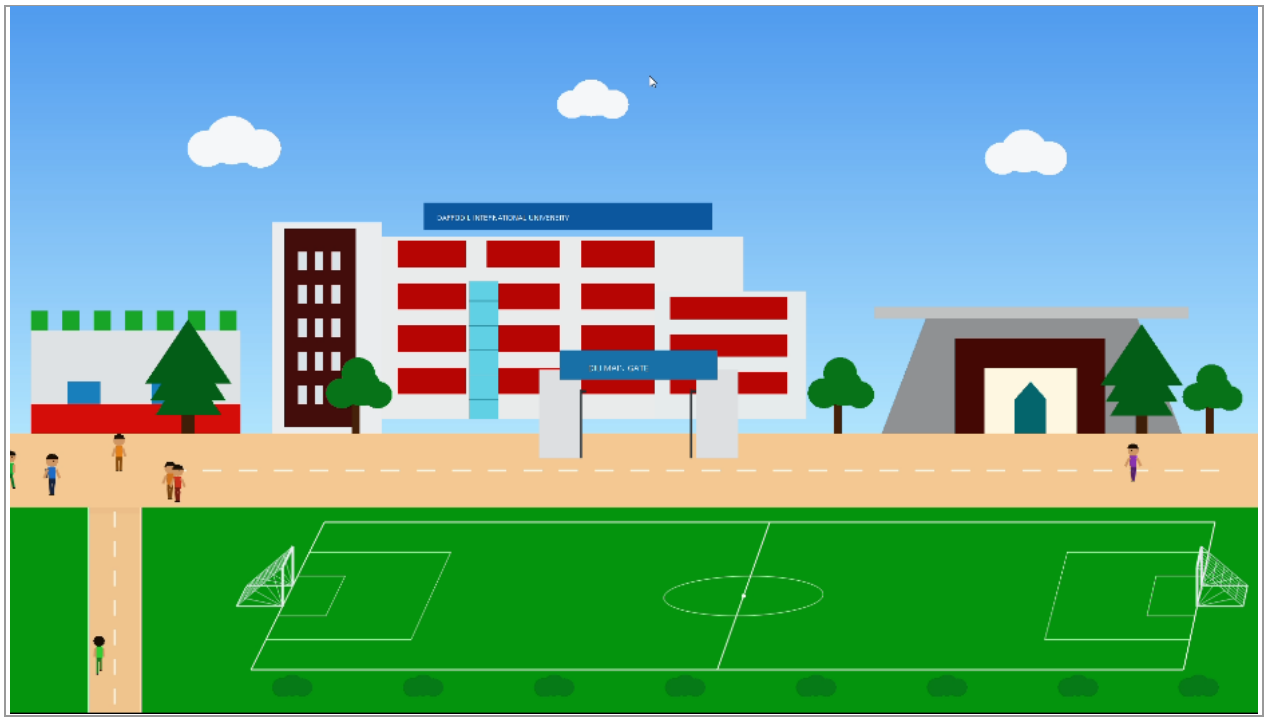


Figure 05b: Scene 4 - Mid Phase

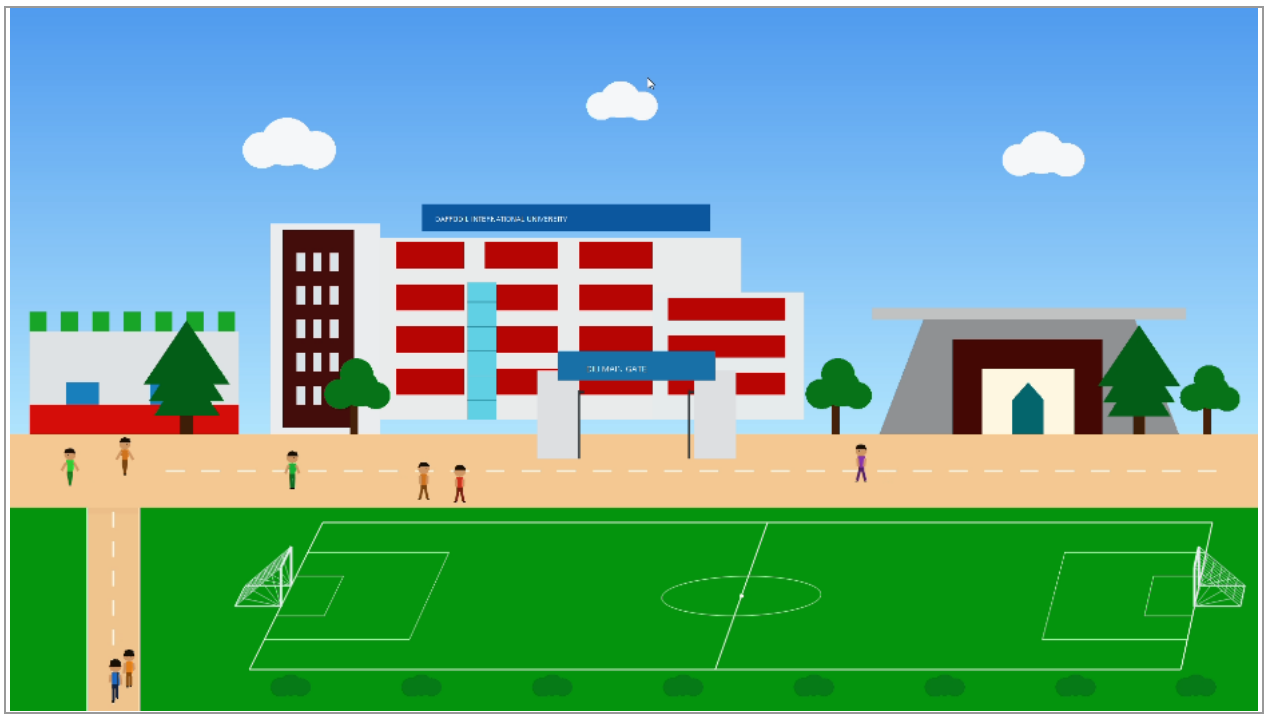


Figure 05c: Scene 4 - End Phase

Scene 5: Elevator Ride to the Classroom Floor

Figure 06a-c below document the three required phases of this scene.



Figure 06a: Scene 5 - Start Phase



Figure 06b: Scene 5 - Mid Phase



Figure 06c: Scene 5 - End Phase

Scene 6: 3D DIU Classroom

Figure 07a-c below document the three required phases of this scene.

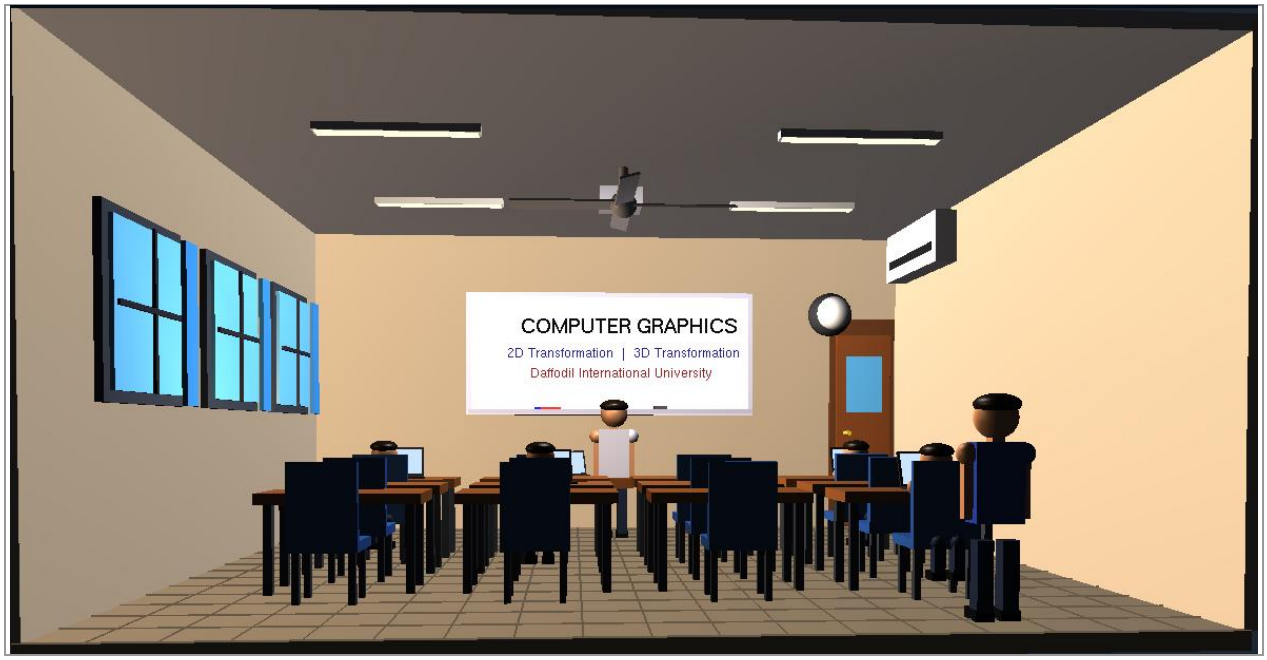


Figure 07a: Scene 6 - Start Phase



Figure 07b: Scene 6 - Mid Phase



Figure 07c: Scene 6 - End Phase

3.3 Discussion

The project successfully fulfils all objectives defined in Chapter 1 across all six scenes.

Strengths

- Consistent, reusable application of the three core rasterization principles (DDA, Bresenham's, Midpoint Circle) across six independently developed scenes
- Effective and varied use of 2D transformations (translation, rotation, scaling, reflection, shear), extended into full 3D transformations in Scene 6
- Smooth, timer-driven real-time animation running at ~60 FPS in every scene
- Modular, well-structured code with a clear, growing library of reusable draw functions per scene
- A meaningful, relatable real-world narrative (a DIU student's daily campus journey) that ties six otherwise separate demos into one coherent project
- A deliberate technical progression from static 2D scenes to a dynamic, lit, camera-controlled 3D classroom

Challenges Faced

- Managing multiple, independently timed animation state machines (per-scene timelines, manual vs automatic control modes) without visual glitches
- Ensuring smooth rendering and consistent frame pacing without flickering, particularly during scene transitions such as door opening/closing and character pose changes
- Balancing object proportions and screen-space layout consistently across six scenes with different subject matter (an interior bedroom, an outdoor street, a moving bus, a campus exterior, an elevator cabin, and a 3D classroom)
- Synchronizing WinMM ambient and event sound effects (bus, traffic, footsteps, classroom ambience, fan) with the corresponding on-screen visual events in Scenes 2, 3, 4, and 6
- Extending the project's 2D primitive-based drawing conventions into genuine 3D solids, lighting, and camera control for Scene 6 without breaking the visual continuity of the overall narrative

Solutions

- Used glutTimerFunc(16, timer, 0) consistently across all scenes to synchronize animation updates at a fixed ~60 FPS interval
- Defined explicit timeline constants (e.g., ALARM_TIME, WAKE_END, WALK_START) and finite-state-machine logic (e.g., the door-close/ascend/floor-update/door-open cycle in Scene 5) to make each scene's phase transitions deterministic and bug-resistant
- Applied glPushMatrix()/glPopMatrix() blocks around glTranslatef/glRotatef/glScalef calls to keep transformations local to each object and prevent unintended cumulative transforms
- Used WinMM's playSoundOnce()/playSoundLoop()/stopAllSounds() helper wrappers to keep sound-event triggering tied directly to the same state variables that drive the visuals

- Organized every scene's code into modular, single-responsibility draw functions, which made it straightforward to test, debug, and later extend each scene independently

Chapter 4

Engineering Standards and Mapping

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

The primary impact of this project on life is educational and demonstrative. By digitizing a familiar daily routine -- waking up, commuting, and attending class at DIU -- into an interactive, real-time animated sequence, the project provides an accessible and engaging medium for students to see abstract computer-graphics concepts (rasterization, transformation, timer-based animation, 3D lighting) applied to a scenario they immediately recognise.

- Skill Development: For the developers, the project refines the ability to translate abstract mathematical concepts (coordinate geometry, trigonometry, matrix-based transformations, and lighting equations) into tangible, moving visual experiences.
- Accessibility: The finished scenes can be run and explored by any student with a Windows machine, letting viewers experience a stylized version of the DIU campus commute without needing to be physically present.

4.1.2 Impact on Society & Environment

Social Impact: This project functions as a small piece of digital storytelling grounded in campus life. By depicting a recognisable DIU student routine -- from the main gate to the academic building to the classroom -- it fosters a sense of familiarity and shared identity among students and demonstrates that engineering coursework can also represent everyday, human experience.

Environmental Impact: As a purely digital creation, this project has a near-zero physical footprint. Unlike a physical scale model built from foam board, wood, or plastic, this OpenGL simulation requires no physical raw materials, produces no waste, and contributes to a paperless engineering workflow. It is an environmentally efficient way to prototype and demonstrate animated scene design.

4.1.3 Ethical Aspects

In this Computer Graphics project, ethical principles were followed to ensure that the design and implementation of every scene is original, respectful, and free from plagiarism or unauthorized content. All visual elements, geometry, colours, and animations were self-created within the source code rather than copied from external assets, maintaining academic integrity and respecting intellectual property rights.

Furthermore, care was taken to ensure that none of the six scenes -- the bedroom, the street, the bus journey, the campus arrival, the elevator, or the classroom -- includes any offensive, misleading, or culturally insensitive content. By adhering to ethical guidelines in design, documentation, and presentation, this project promotes creativity, honesty, and professionalism, which are essential values in responsible engineering and digital content creation.

4.1.4 Sustainability Plan

- Code Portability: The project is written in standard C++ with OpenGL/GLUT, so it can be recompiled on Windows, Linux, or macOS (as reflected by the #ifdef __APPLE__ branch retained in Scene 1) with minimal changes, reducing the risk of the codebase becoming obsolete as hardware and operating systems evolve.
- Modularity for Scalability: Each scene is built from small, single-purpose draw functions (e.g., drawSun(), drawBus(), drawElevatorDoors(), drawTeacherTable()). This modular architecture allows future students to "plug in" new features -- such as additional scenes, richer 3D geometry, or new sound effects -- without rewriting the existing core logic.
- Documentation: Inline comments describing each scene's controls, timeline constants, and structure (as seen at the top of every main.cpp file) ensure that the logic behind the animation state machines and rasterization helper functions remains understandable to future students, supporting a cycle of continuous learning and improvement.

4.2 Complex Engineering Problem

4.2.1 Mapping of Program Outcome

This section maps the project's problem and solution approach to the targeted Program Outcomes (POs).

Table 4.1: Justification of Program Outcomes

POs	Justification of Mapping (Project Perspective)
PO1	This project applies mathematical and computational knowledge -- coordinate geometry, trigonometric motion equations, and matrix-based model-view transformations -- to construct and animate graphical objects across six OpenGL scenes, integrating rasterization theory with real-time rendering to solve a complex, multi-scene visualization and animation problem. (PO1)
PO5	The project incorporates modern computer graphics tools and technologies -- OpenGL, FreeGLUT, the WinMM audio API, and Code::Blocks -- to ensure the application of contemporary engineering practices and real-time visualization techniques across all six scenes. (PO5)
PO9	Through collaborative project work spanning six independent scenes, every team member was engaged in teamwork, task allocation (one or more scenes per member), integration, and joint documentation of the final report. (PO9)
PO10	The project required presenting the developed six-scene animation system based on the gathered narrative and technical requirements, preparing this technical report, and communicating design decisions, algorithms, and transformations clearly. (PO10)

4.2.2 Complex Problem Solving

Table 4.2: Mapping with complex problem solving.

EP1 Depth of Knowledge	EP2 Range of Conflicting Requirements	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent of Stakeholder Involvement	EP7 Interdependence
✓		✓		✓		

EP1: This project requires the team to design, implement, and render six distinct 2D and 3D graphical scenes using OpenGL. These tasks involve applying rasterization principles, geometric transformation logic, and timer-driven animation techniques, which together develop strong analytical and problem-solving skills in real-time visualization and rendering.

EP3: In this project, the team analyzes object structures, transformation sequences, timeline/state-machine boundaries, and animation timing across six scenes to create smooth and believable motion. By breaking each complex scene into smaller modular draw functions and managing their interactions and transitions, the project strengthens the team's ability to formulate and analyze complex graphical problems effectively.

EP5: The team applies established graphics principles and transformation code -- including DDA, Bresenham's, Midpoint Circle, and matrix-based operations -- consistently across all six scenes to implement optimized, real-time rendering. This hands-on application of standard programming and algorithmic conventions in computer graphics enhances the team's ability to conduct investigations using established graphics methods and tools.

4.2.3 Complex Engineering Activities

Table 4.3: Mapping with complex engineering activities.

EA1 Range of Resources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for Society and the Environment	EA5 Familiarity
✓	✓	✓		

EA1: In this Computer Graphics project, the team follows systematic methods to design and document all six scenes' graphical components, algorithms, and transformations. The project structure, rendering logic, and animation workflow are organized clearly in the source code and in this report, ensuring that the technical information is logically presented and easily understood by the reader.

EA2: For this project, the team uses written and visual presentation techniques -- including this structured technical report and the required start/mid/end phase screenshots for every scene -- to explain complex computer graphics concepts such as transformations, animation logic, and rendering processes, enhancing communication skills and the ability to convey design decisions clearly.

EA3: The project demonstrates innovation by chaining six independently themed scenes into one continuous narrative and by deliberately escalating technical scope -- from static 2D orthographic

rendering in Scene 1 to a fully lit, camera-controlled 3D classroom in Scene 6 -- going beyond the scope of a single-scene lab exercise.

Chapter 5

Conclusion

5.1 Summary

The "A Day at DIU" project successfully achieved its goal of creating a technically rigorous and narratively meaningful sequence of six real-time animated scenes using OpenGL and GLUT. By consistently applying the classical rasterization principles -- DDA, Bresenham's, and the Midpoint Circle Algorithm -- through a shared set of drawing utilities, the project demonstrates the mathematical foundation required for computer graphics across a wide range of visual contexts: an interior bedroom, an outdoor street, a moving bus, a university campus, an elevator cabin, and a fully lit 3D classroom.

The project culminates in a complete narrative arc following a DIU student's day, featuring:

- A morning wake-up sequence with a timeline-driven character pose transition and an interactive alarm clock (Scene 1)
- A street-level commute scene with traffic, pedestrians, and synchronized ambient sound (Scene 2)
- A parallax-scrolling bus journey across a river and bridge into the city (Scene 3)
- Arrival at the DIU campus with a crowd of independently phased walking students (Scene 4)
- An elevator ride modeled as an explicit door-close/ascend/floor-update/door-open state machine (Scene 5)
- A fully lit, camera-controlled 3D classroom built from cuboid, cylinder, and sphere primitives (Scene 6)

Ultimately, the project serves as a bridge between everyday student experience and rigorous technical application, meeting all academic requirements for 2D and 3D transformations, rasterization principles, and real-time rendering.

5.2 Limitations

- Fixed Orthographic Projection: Scenes 1-5 are rendered in a strictly 2D orthographic coordinate system (glOrtho). While depth is simulated through scaling and layering, these scenes lack a true 3D Z-axis or free camera rotation, which only Scene 6 introduces.
- Basic Interaction: User interaction is largely limited to discrete keyboard triggers (stage-jump keys, pause/restart, alarm/fan toggles) and, in Scene 1, a single mouse click target. There is no free-roam camera or drag-based manipulation across the 2D scenes.

- Non-Physics-Based Motion: Character walking, bird flight, cloud drift, and vehicle movement are based on simple linear interpolation and sine-wave functions rather than true physics simulation (no collision response, momentum, or inverse kinematics).
- Platform Dependency for Audio: Because sound in Scenes 2, 3, 4, and 6 relies on the Windows Multimedia API (WinMM/PlaySoundA), the audio layer of those scenes is Windows-specific and would need to be replaced with a cross-platform library to run identically on macOS or Linux.
- Hardware-Defined Resolution: Each scene targets a fixed window size (for example 1200 x 700 in Scene 1) with a hardcoded orthographic projection; resizing the window is handled via letterboxing in the reshape() callback rather than a fully resolution-independent layout.

5.3 Future Work

- Full 3D Transition: Extending Scenes 1-5 from gluOrtho2D toward a perspective projection (gluPerspective) so that the entire six-scene story, not just Scene 6, can be viewed and explored from different camera angles.
- Texture Mapping: Replacing solid-colour fills with real brick, grass, road-asphalt, and fabric textures to enhance visual realism, particularly for the DIU campus buildings and the classroom furniture.
- Advanced Shading Models: Extending the Phong-style lighting already used in Scene 6 to the rest of the project, and introducing Gouraud or fully programmable (shader-based) shading with dynamic light sources such as a moving sun that casts real-time shadows.
- Interactive HUD and Scene Selector: Adding an on-screen menu that lets the viewer jump directly between the six scenes, replay a specific scene, or toggle a "Day/Evening" lighting mode.
- Cross-Platform Sound Integration: Replacing the WinMM-based audio layer with a cross-platform audio library (e.g., OpenAL or SDL_mixer) so that the ambient and event sound effects in Scenes 2, 3, 4, and 6 work identically on Windows, macOS, and Linux.
- Seamless Scene Transitions: Merging the six standalone executables into a single application with automatic scene-to-scene transitions (e.g., a fade or wipe effect) to present the full narrative as one continuous run rather than six separate programs.

Chapter 6

Hand Drawing Graph

Scene 1:

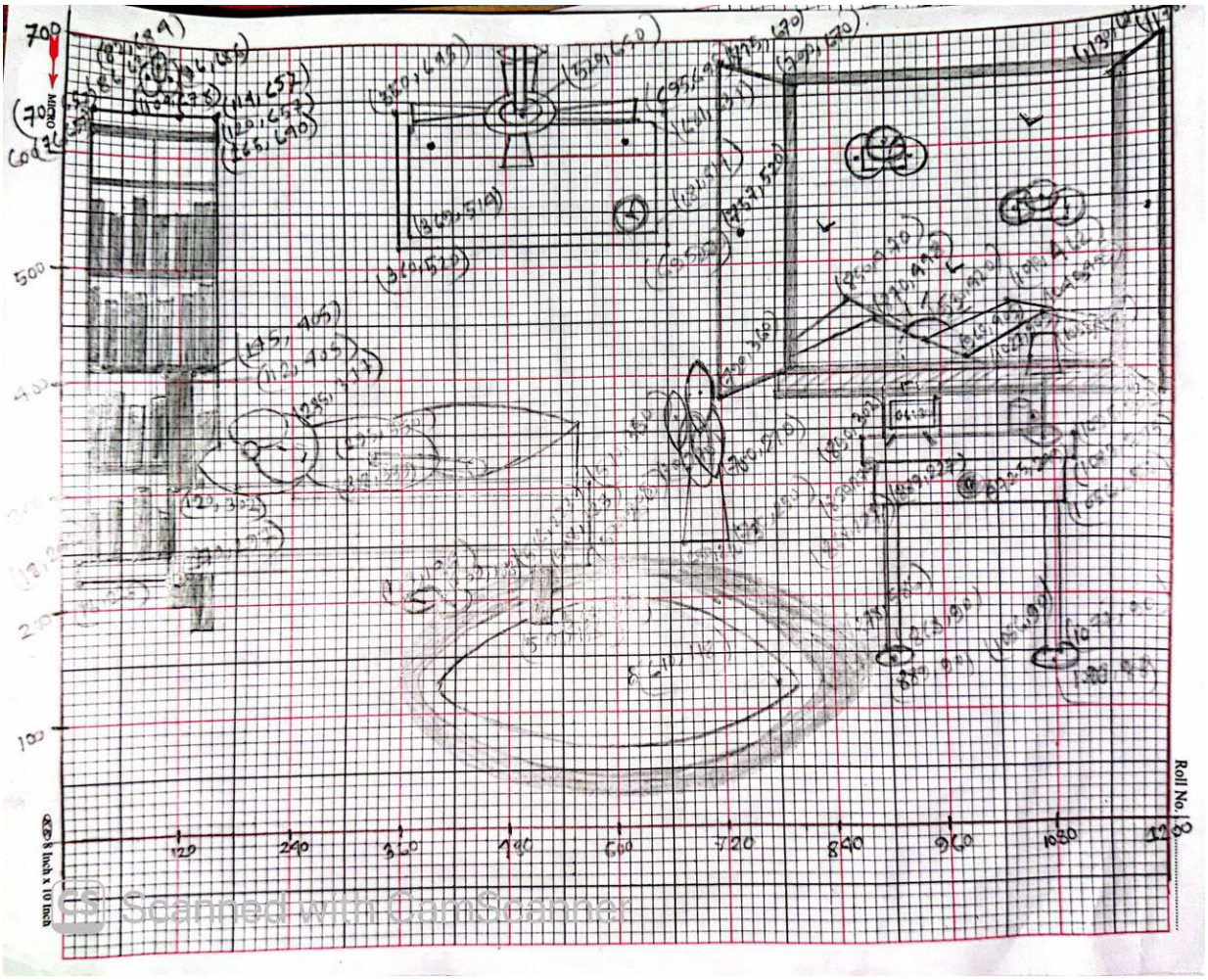


Figure 1.1

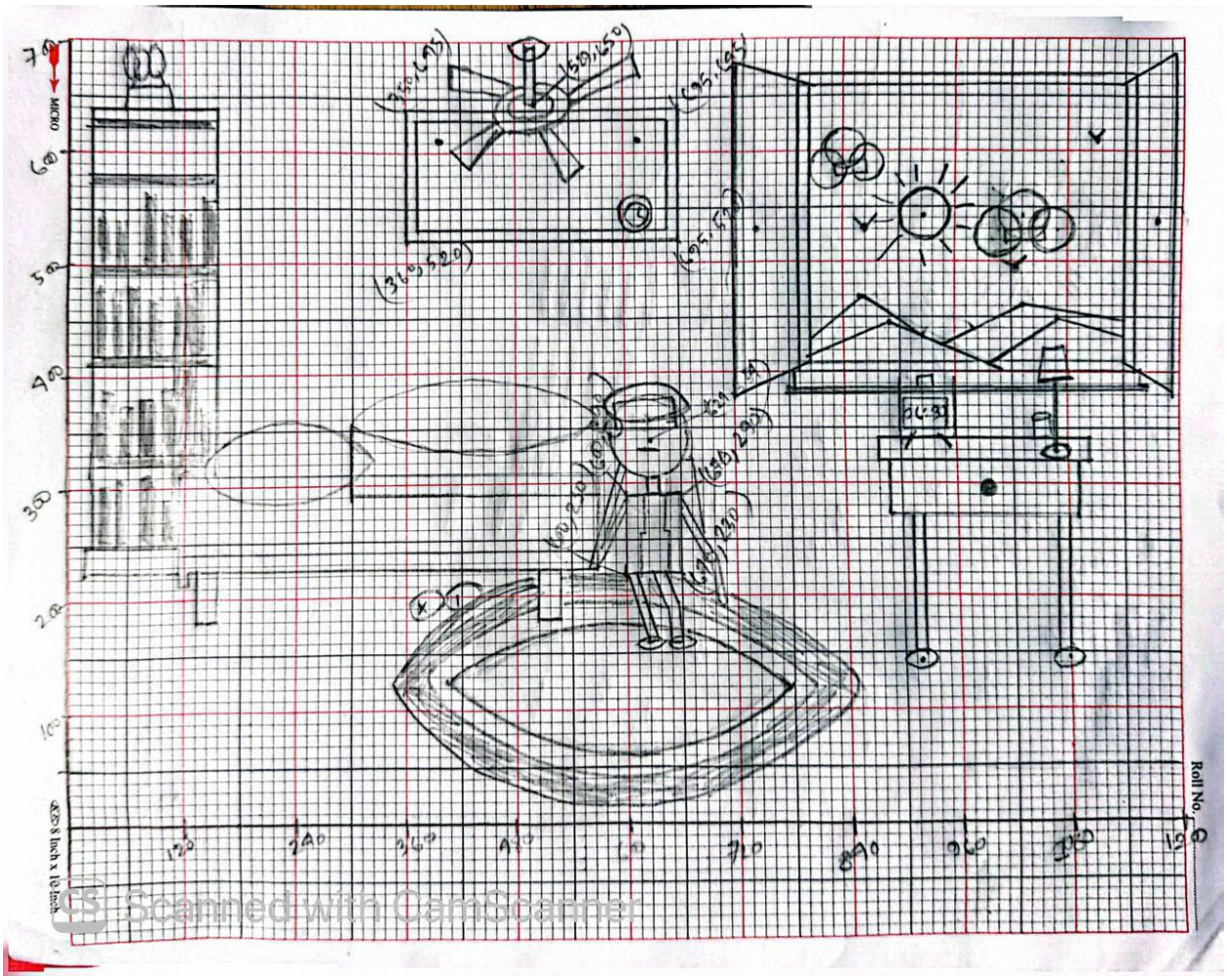


Figure 1.2

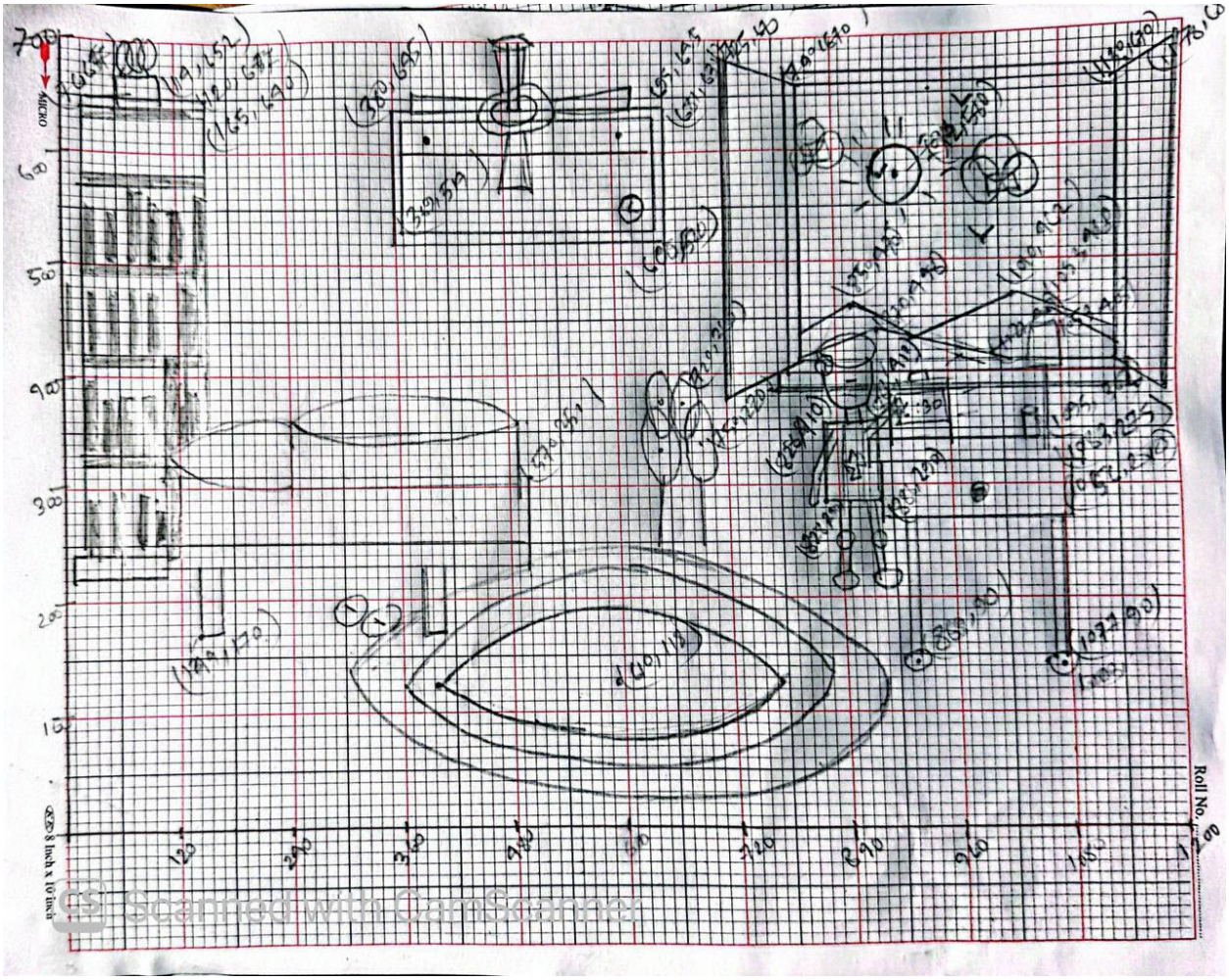


Figure 1.3

Scene 2:

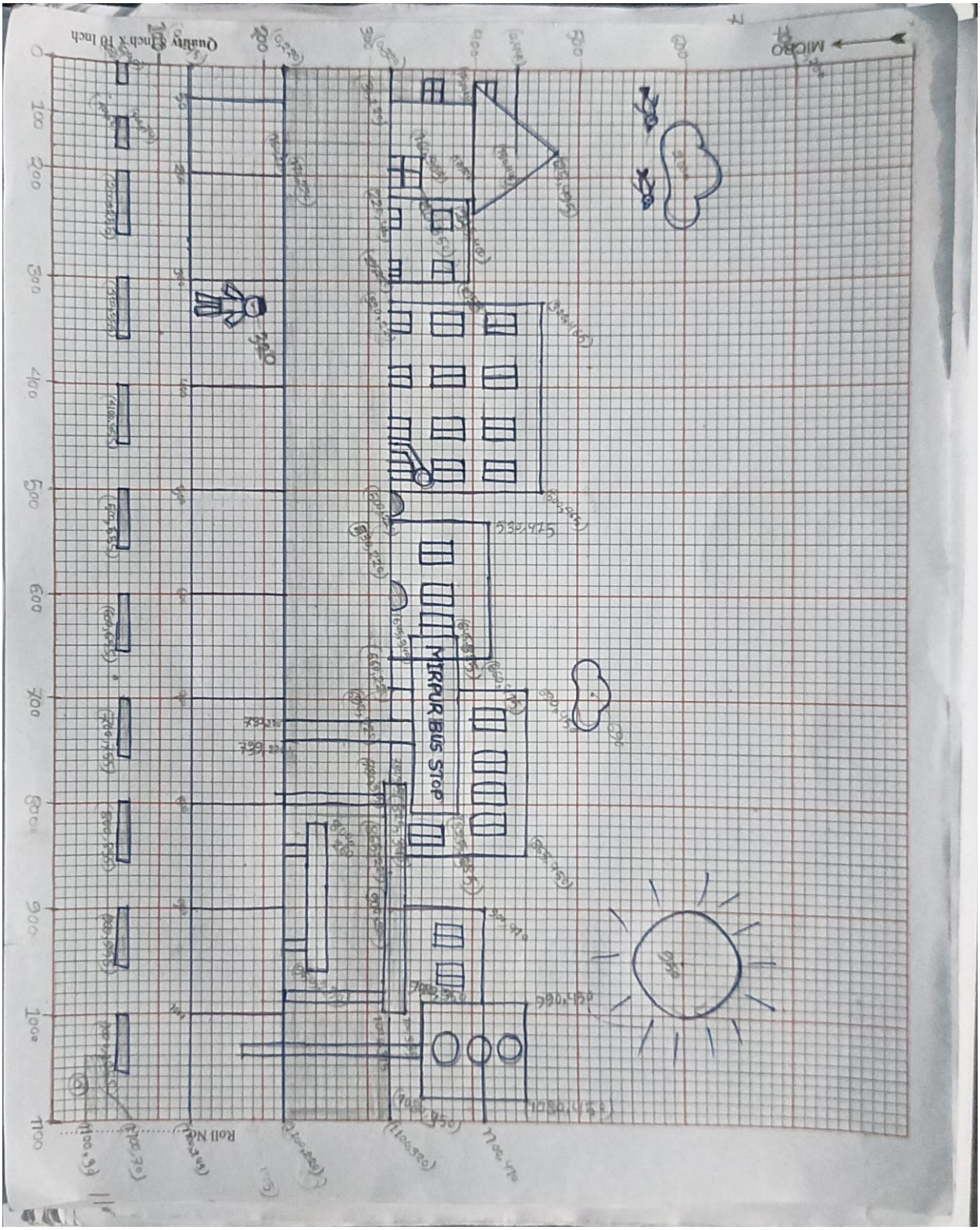


Figure 2.1

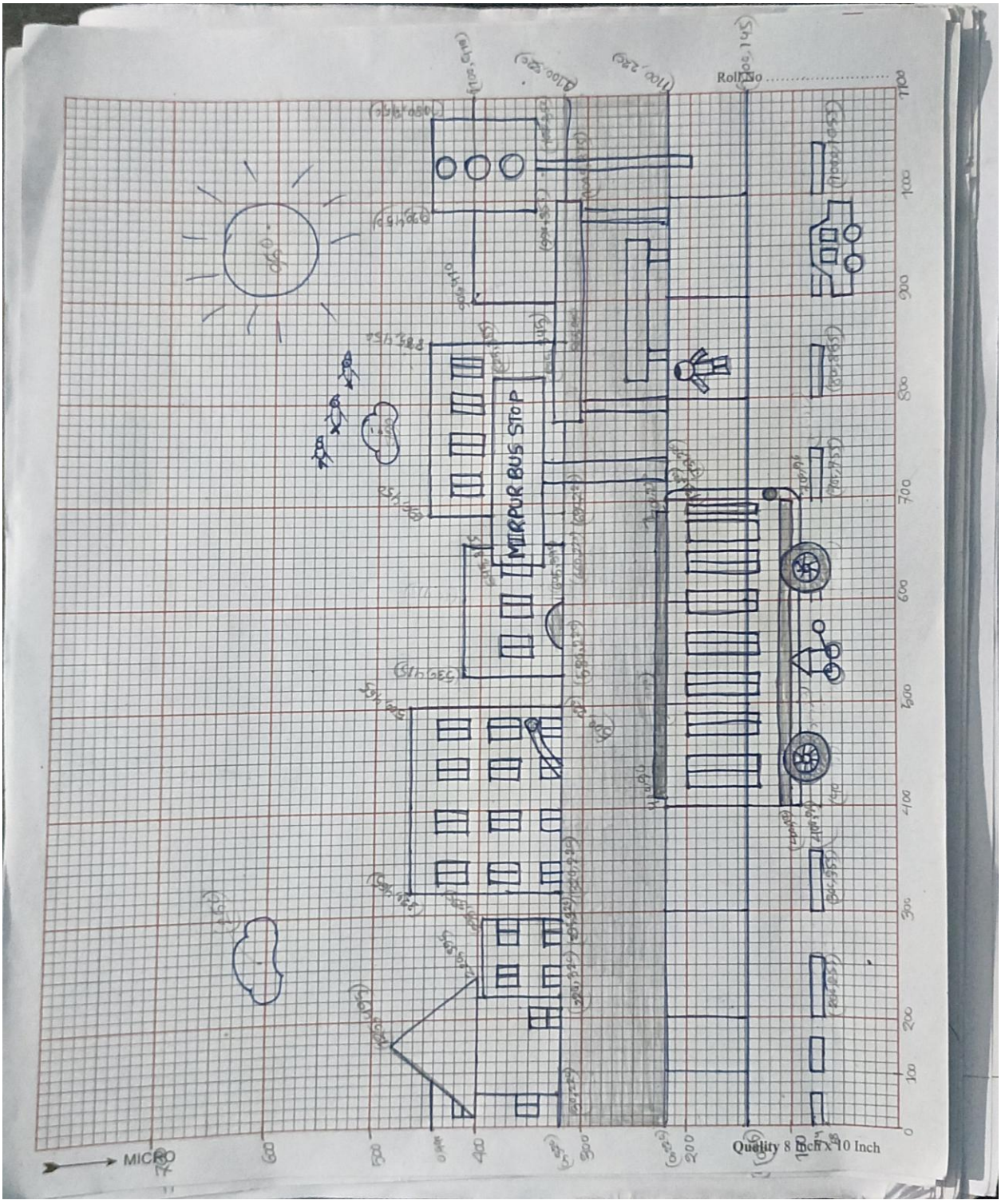


Figure 2.2

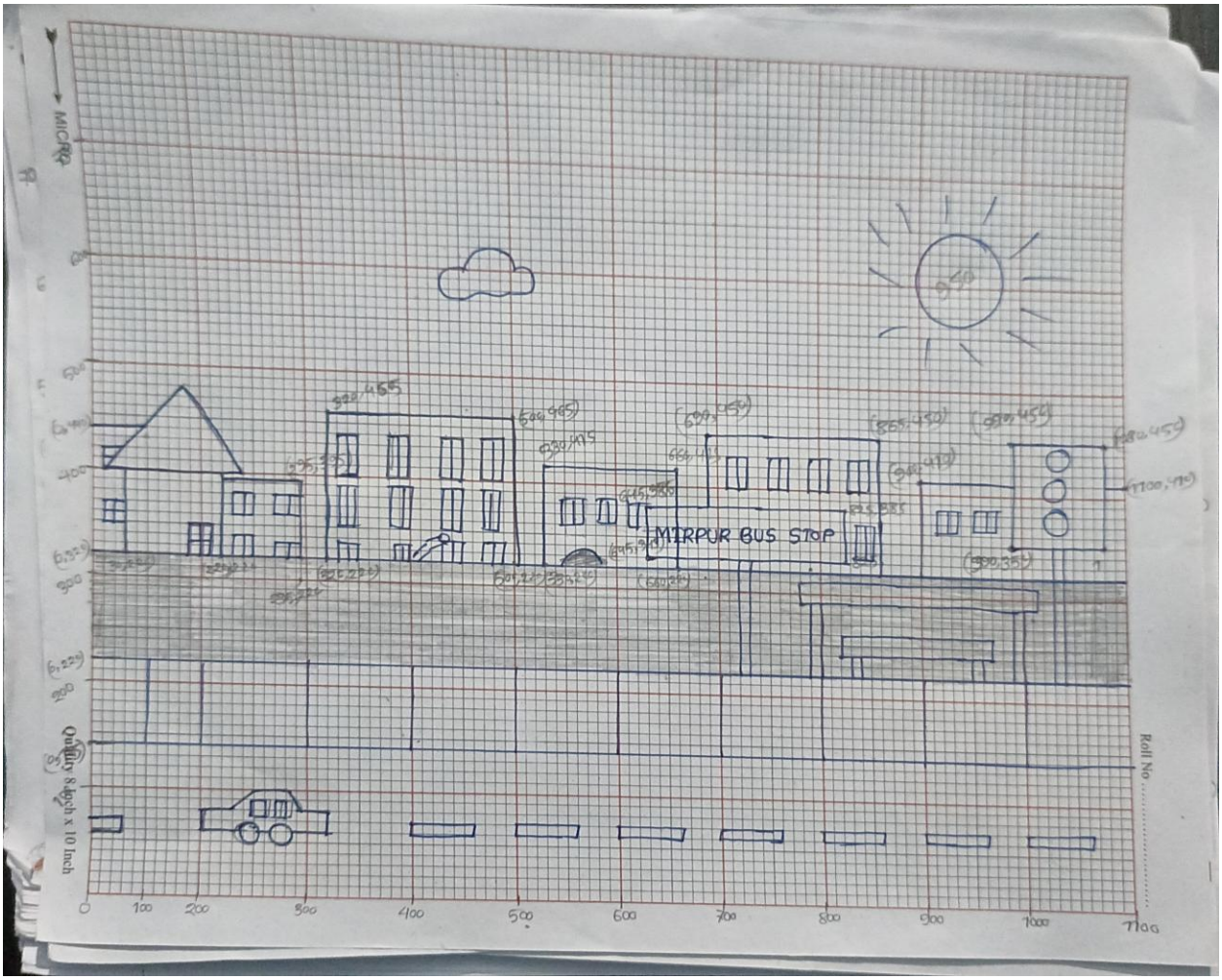


Figure 2.3

Scene 3:

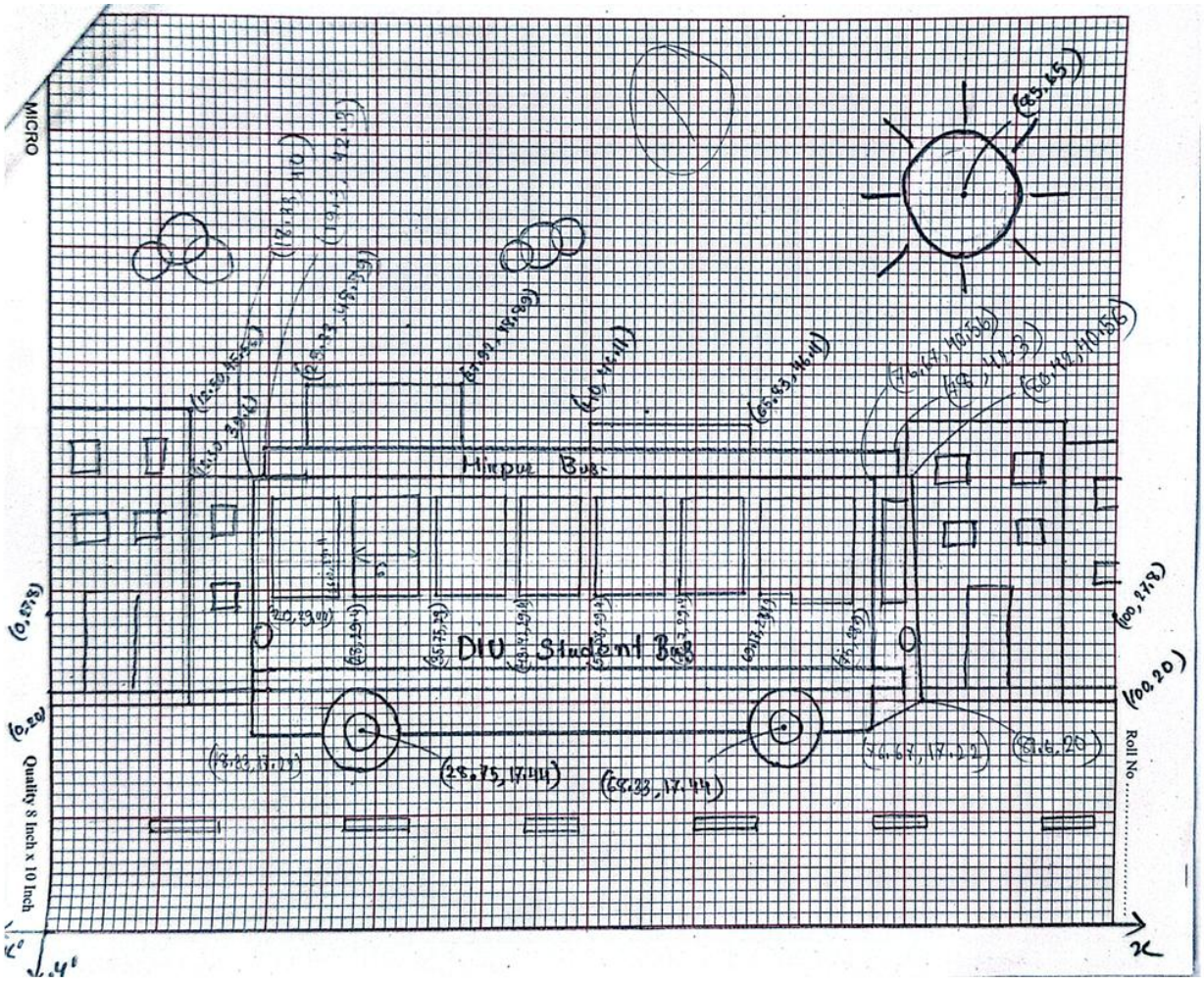


Figure 3.1

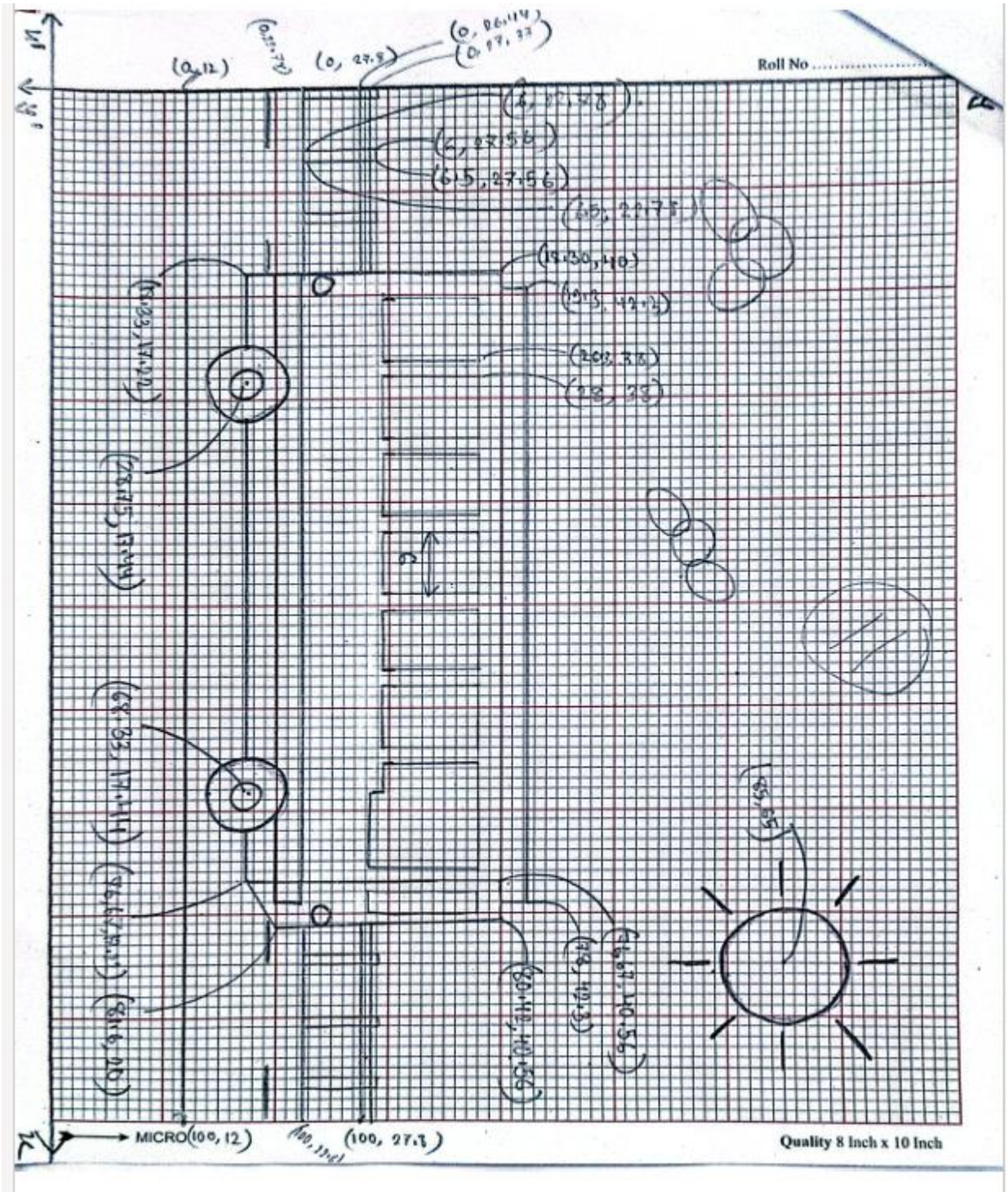


Figure 3.2

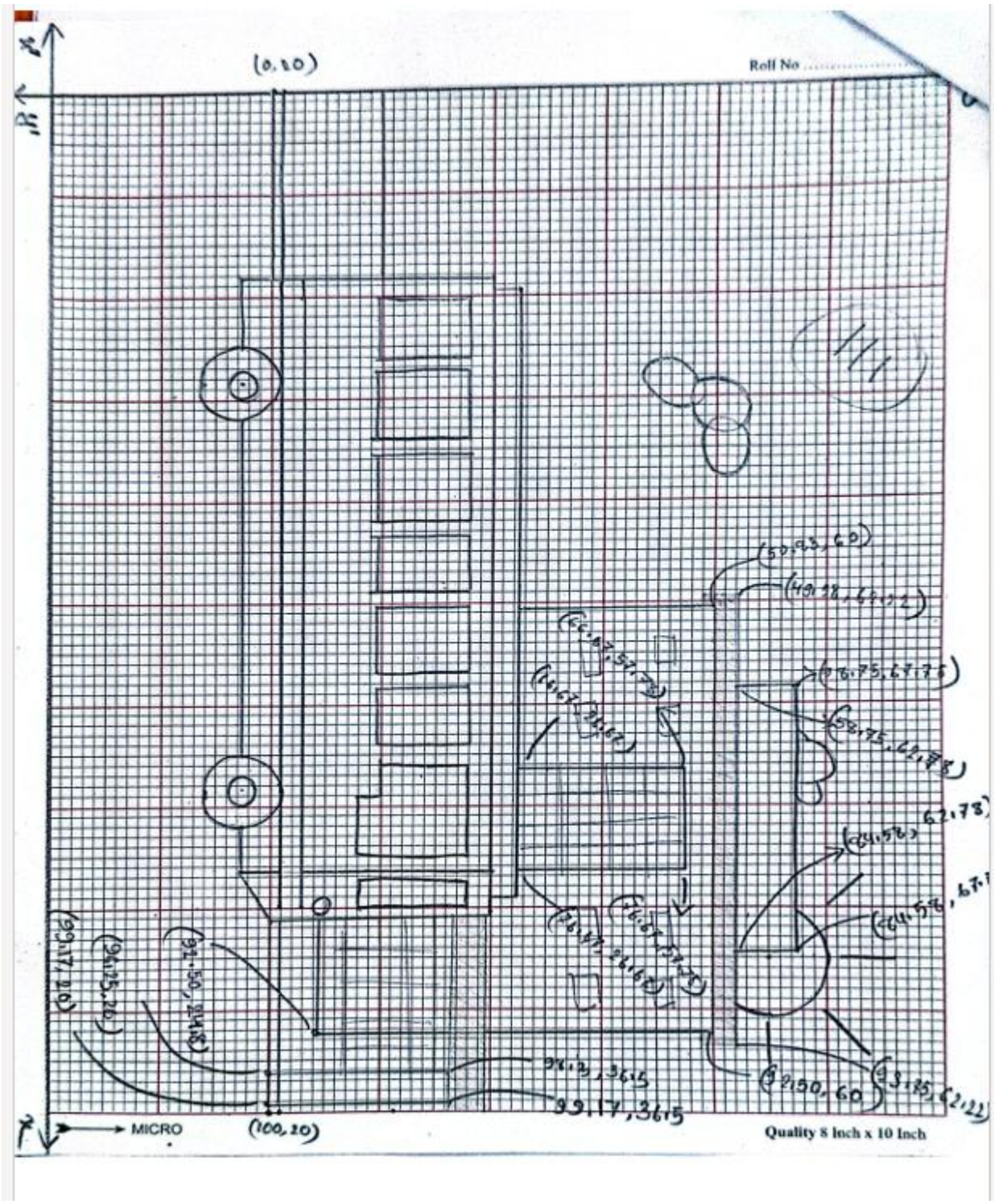


Figure 3.3

Scene 4:

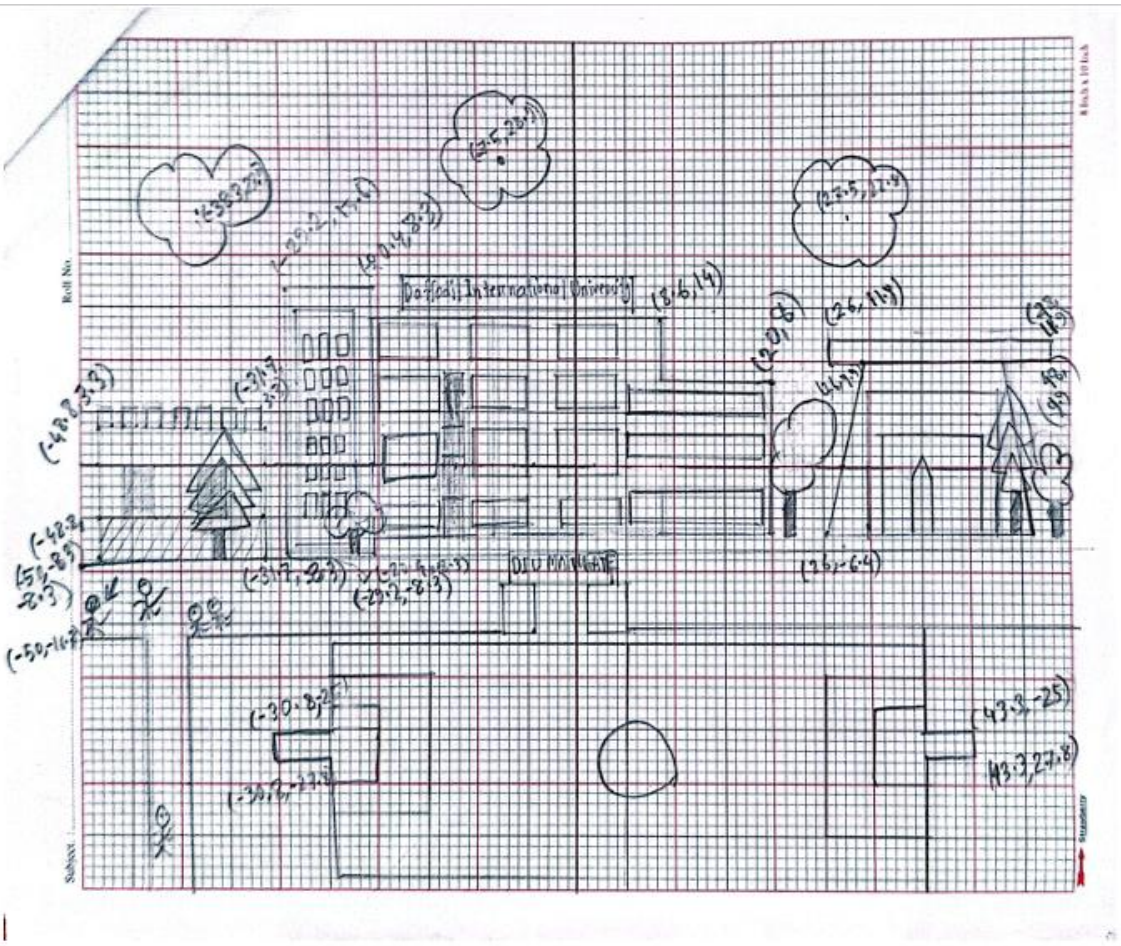


Figure 4.1

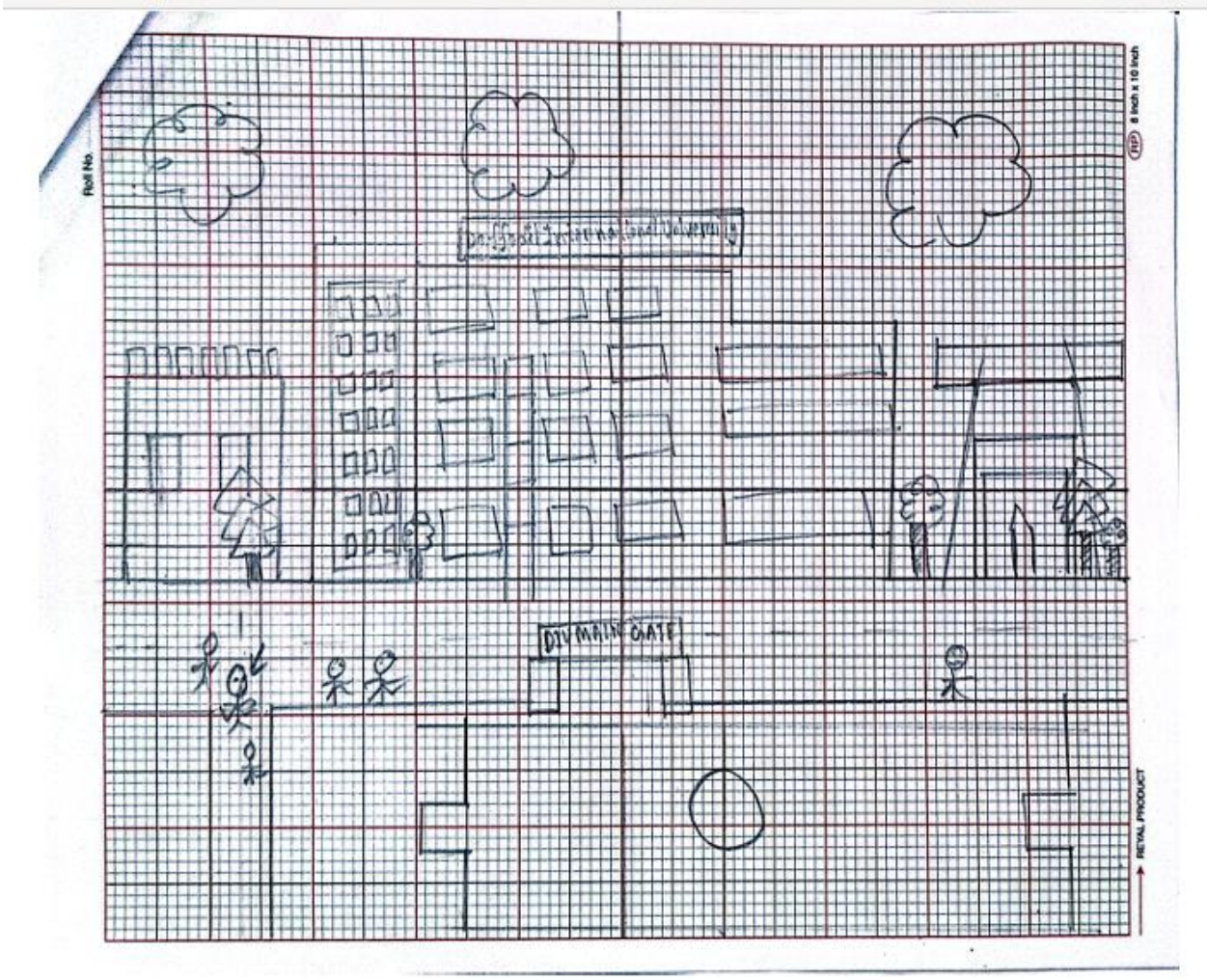


Figure 4.2

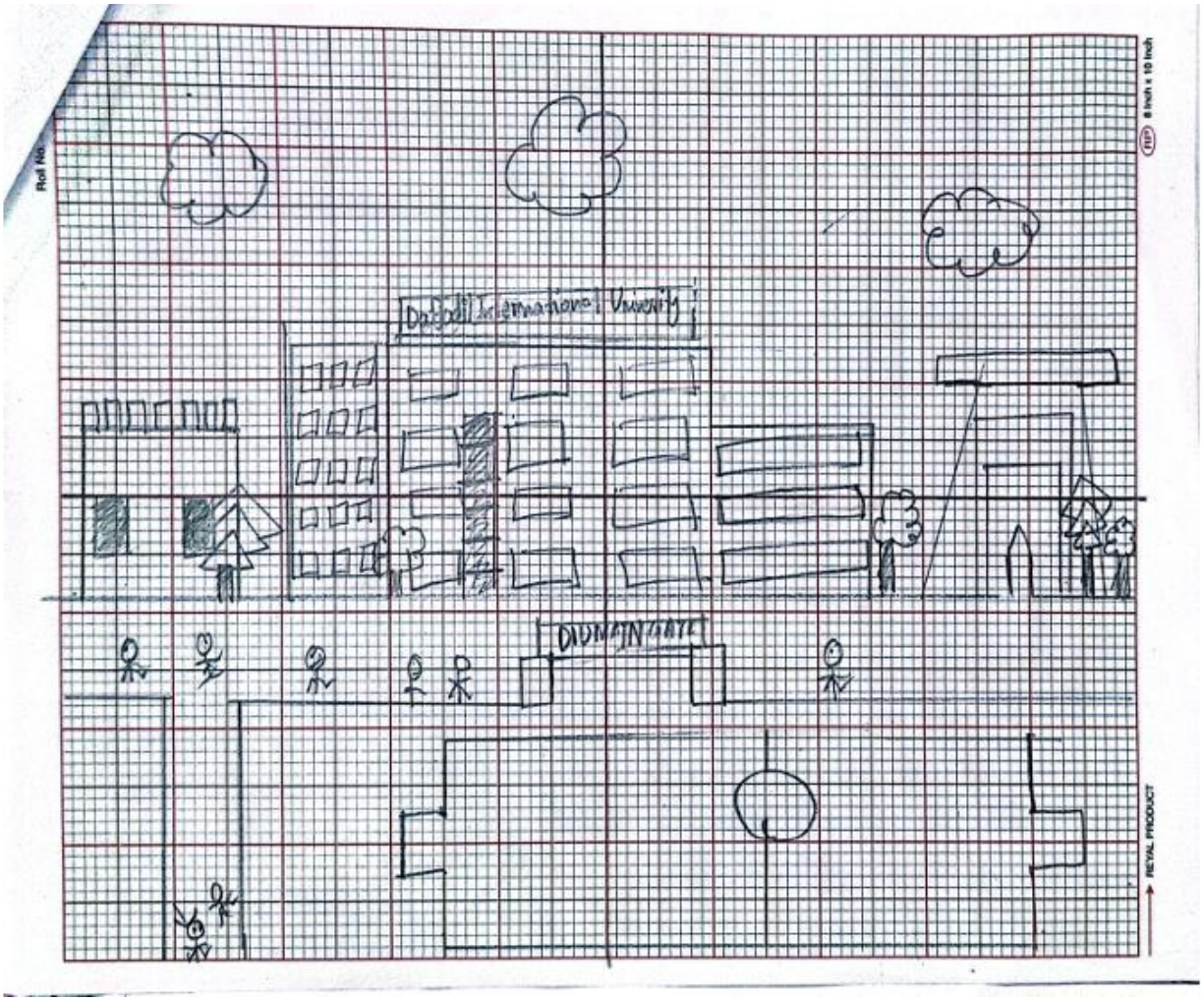


Figure 4.3

Scene 5:

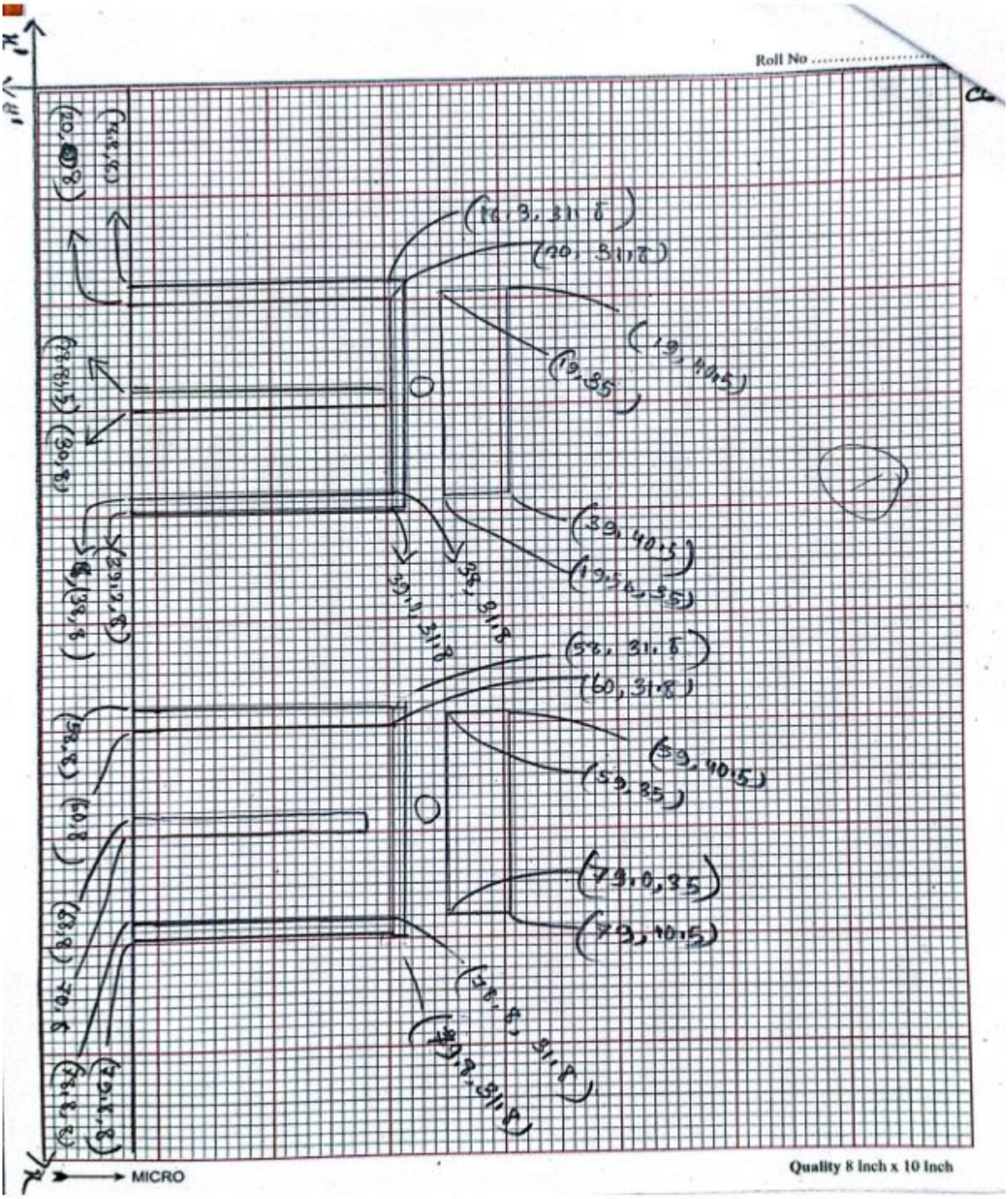


Figure 5.1

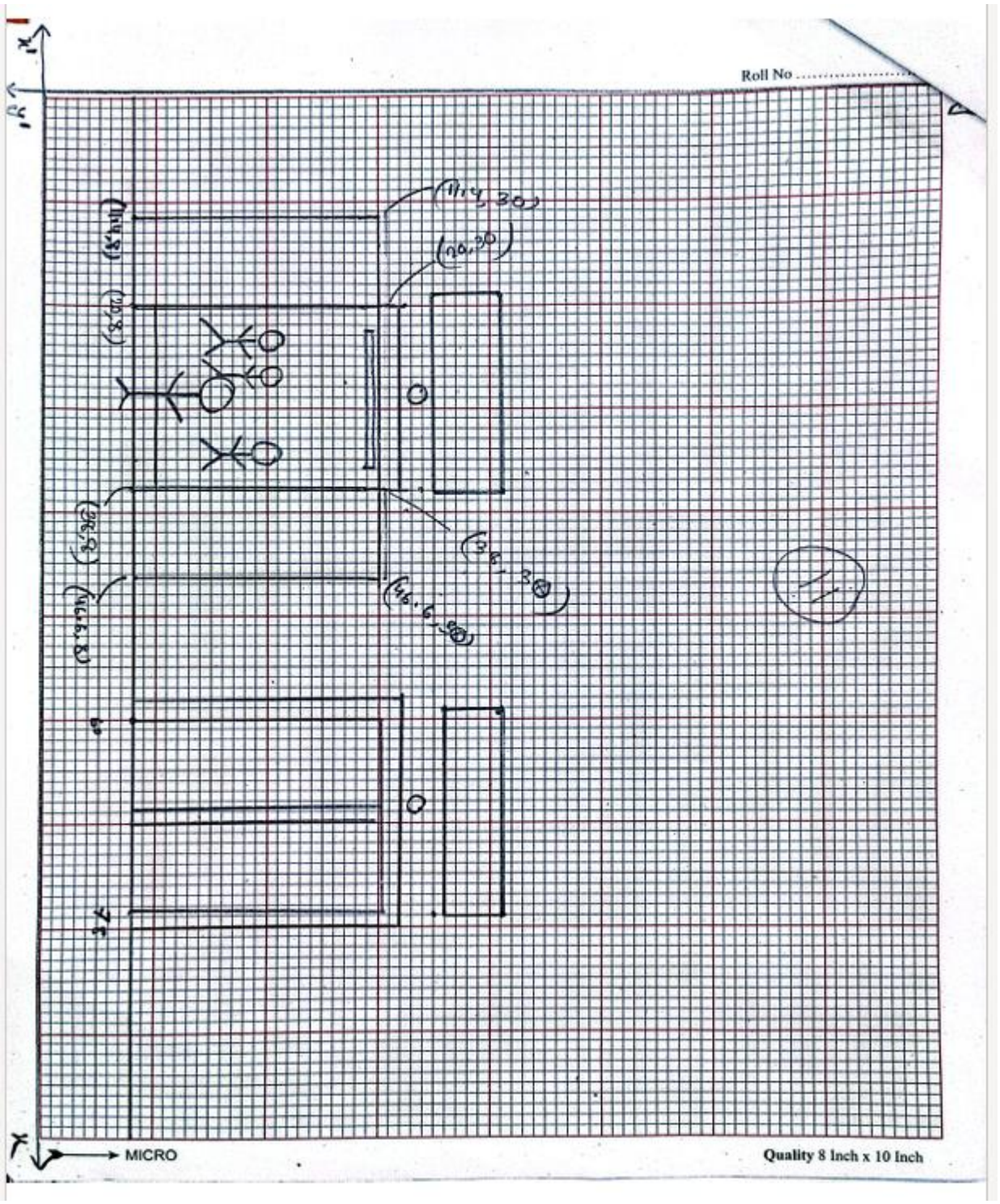


Figure 5.2

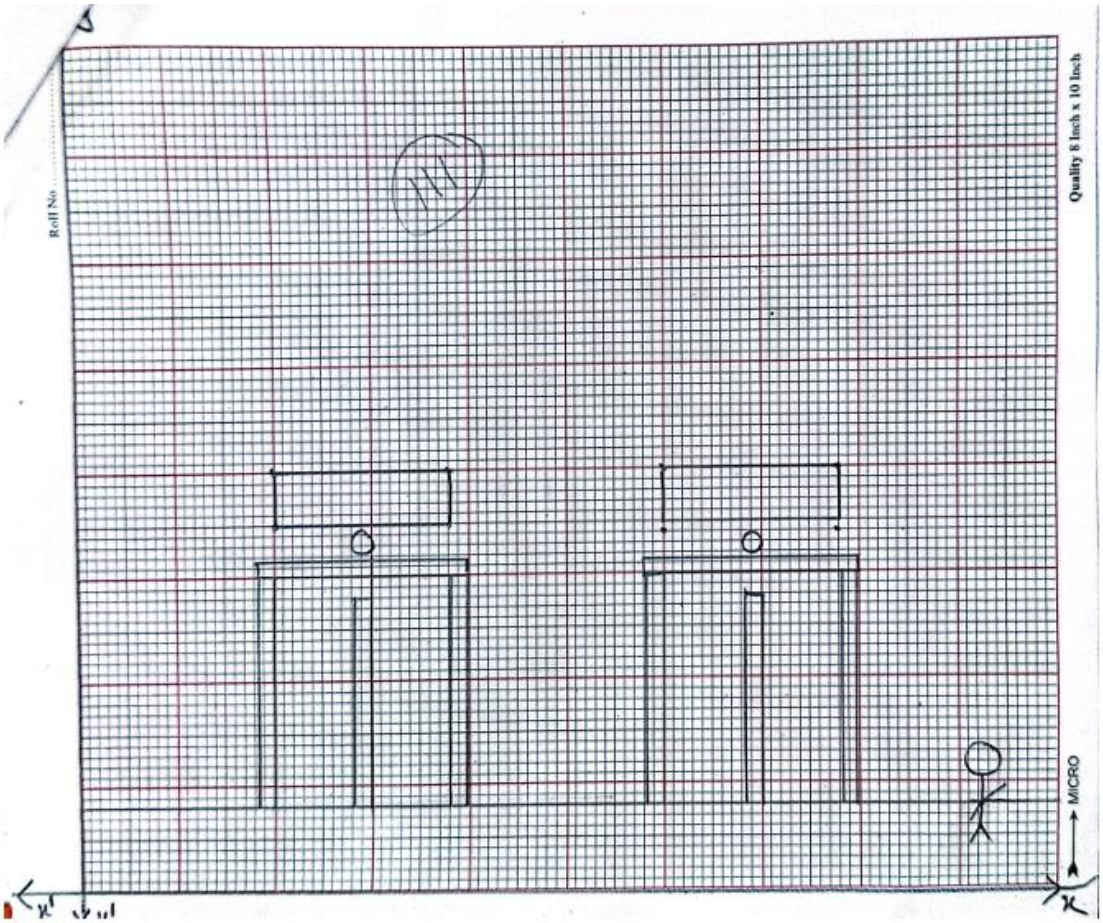


Figure 5.3

Scene 6:

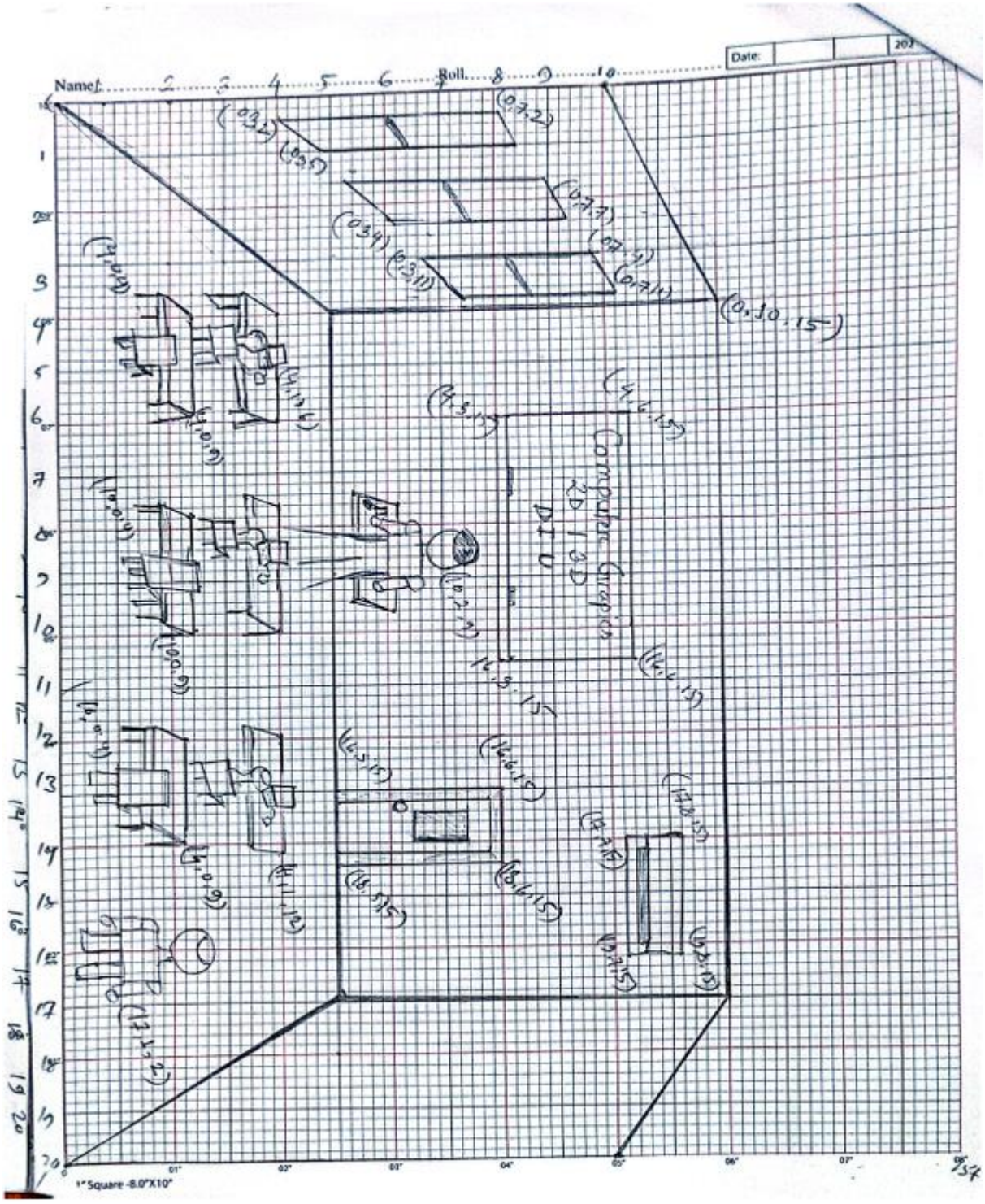


Figure 6.1

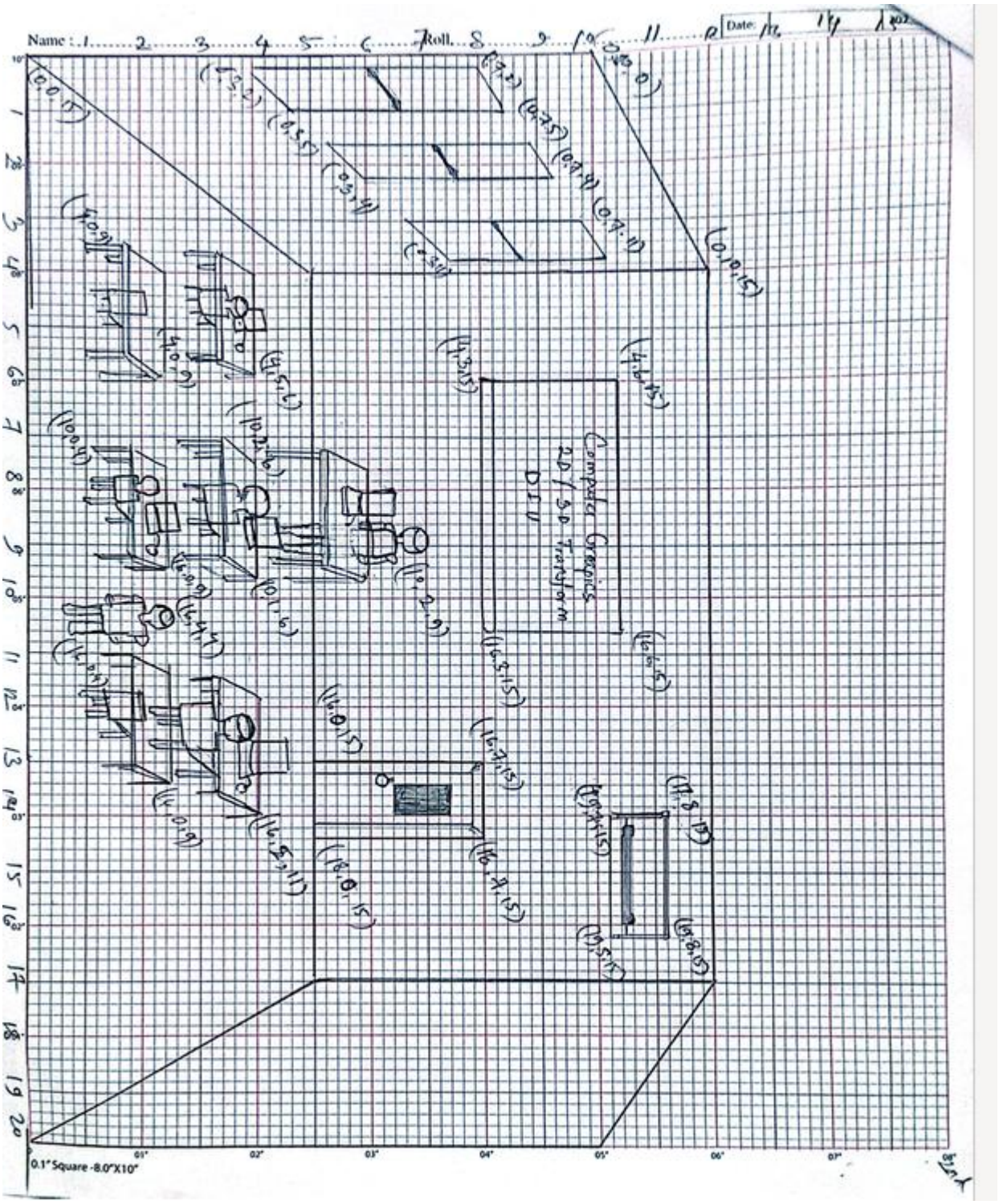

Name :
Roll.
Date:
Computer Graphics
2D / 3D Transform
DSU
0.1" Square -8.0"X10"

Figure 6.2

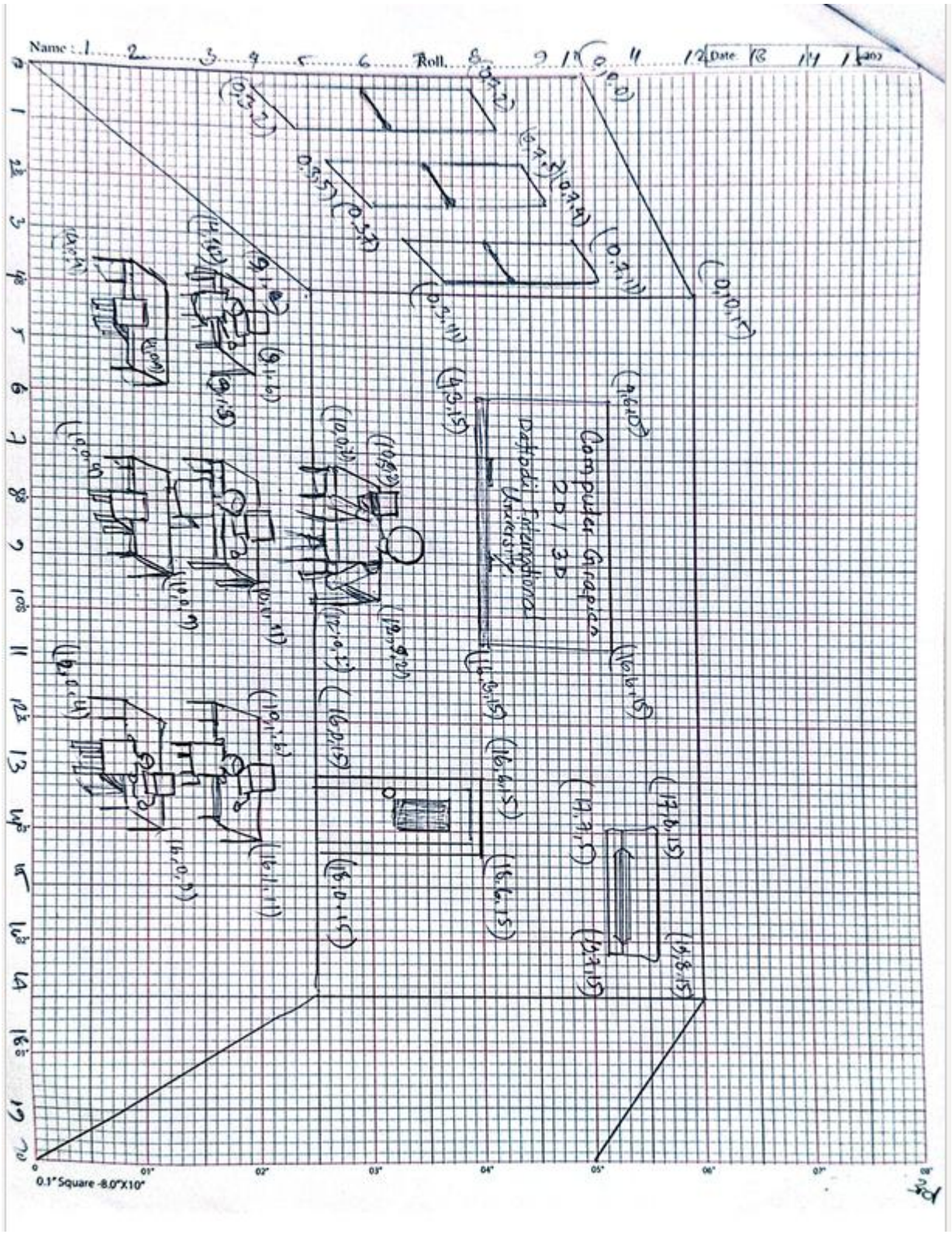

Name :
Roll.
Date
Computer Graphics
2D / 3D
Pathodia Internationol
University
0.1" Square -8.0"X10"

Figure 6.3